

计算机组成原理

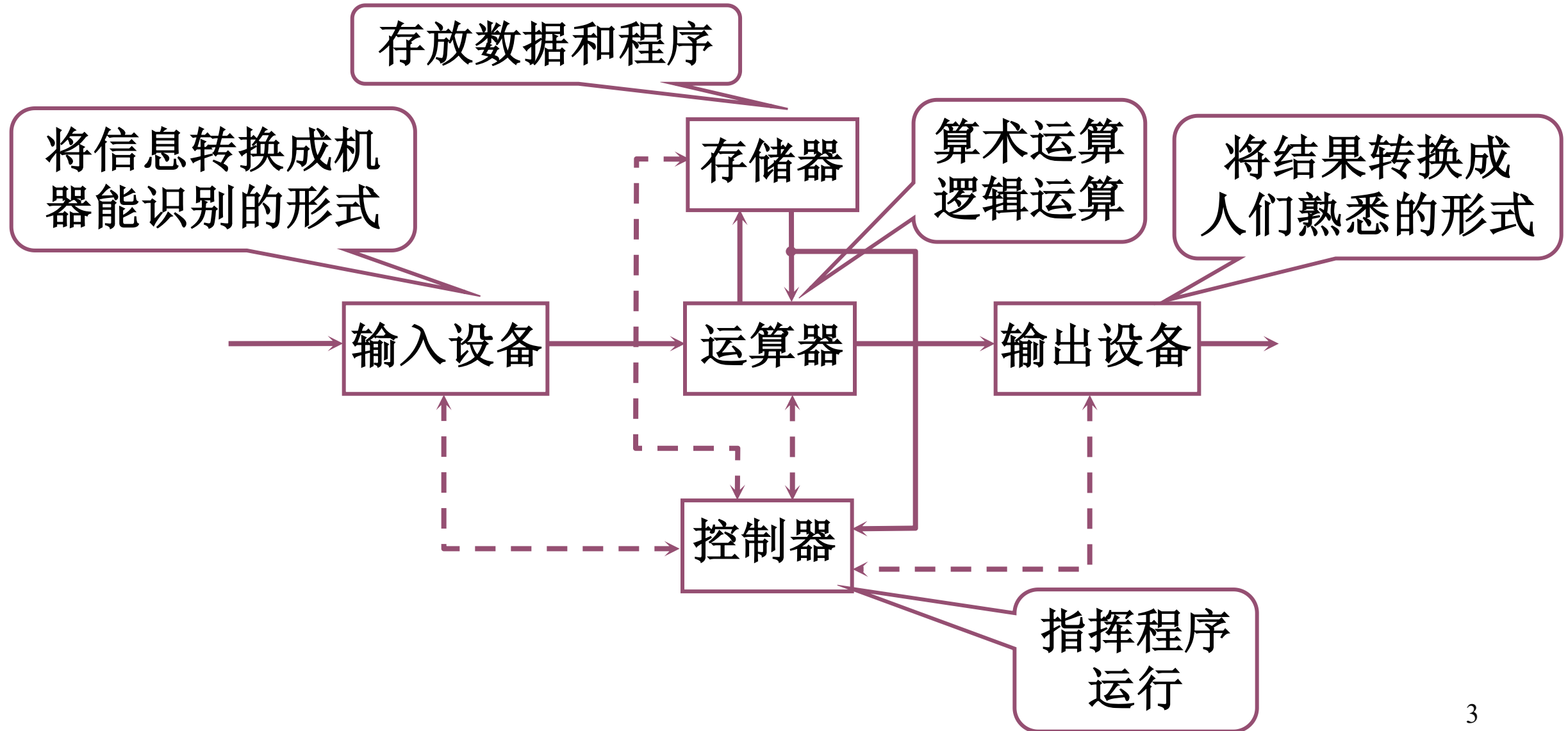
王鸿鹏 wanghp@hit.edu.cn

计算机科学与技术学院

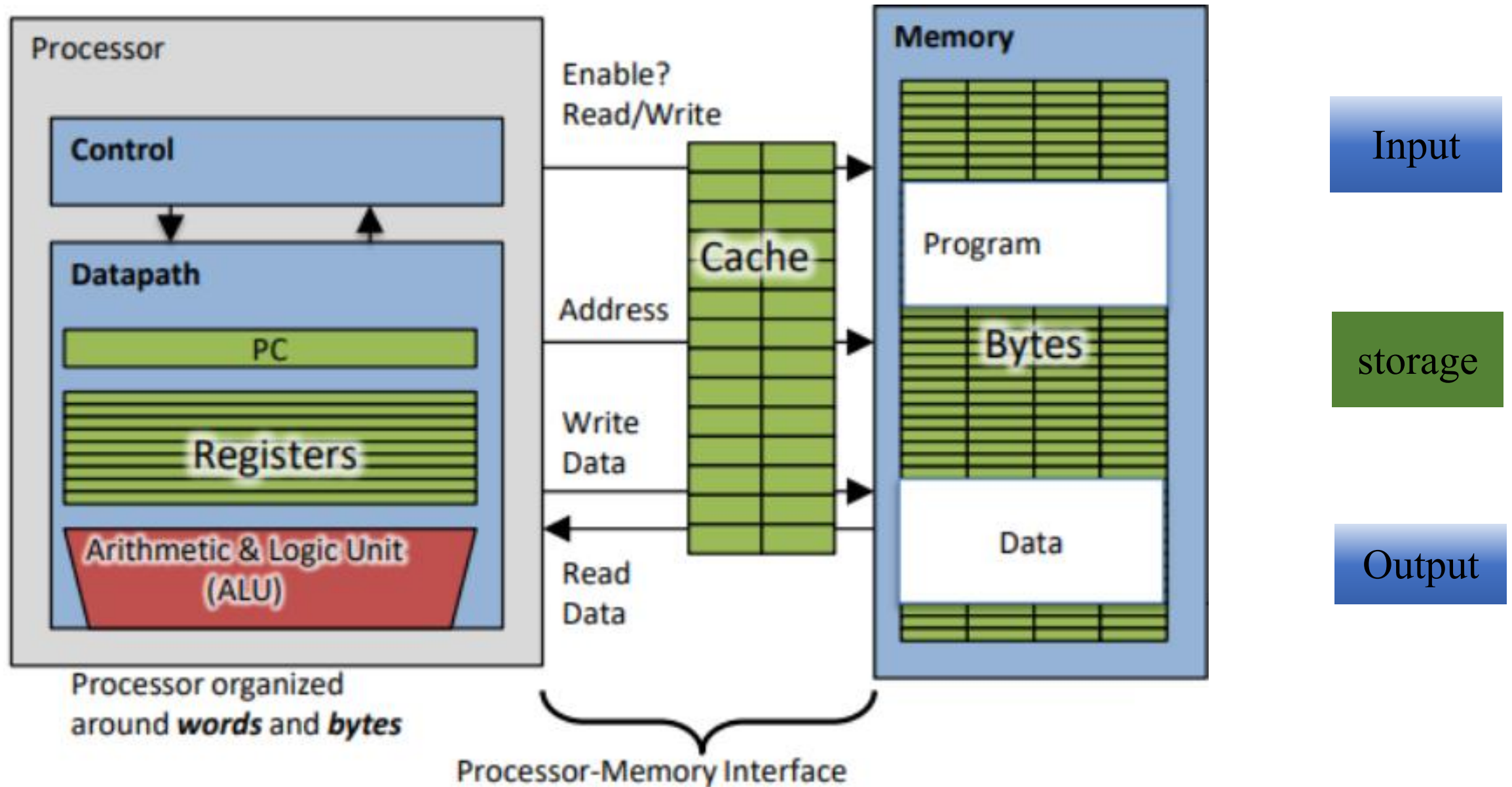
第4章 处理器(Central Processing Unit)设计

- 处理器设计的需求分析
- RISC-V数据通路的组件选择
- RISC-V部分指令的数据通路设计
- RISC-V控制器

冯·诺依曼架构计算机硬件示意图



单核计算机系统逻辑示意图



功能层面上CPU的定义

- CPU(Central Processing Unit): 能够通过输入的机器码, 执行相应操作、并保持相应状态的**数字电路**。

RISC-V机器码 **0x00600293**
(例如: `addi t0, x0, 6`)

数字电路
(逻辑门、寄存器)

执行完这条指令后,
寄存器t0的值为6

t0



处理器（CPU）的组成部分

- **数据通路(datapath / ALU...)**是处理器中执行操作的硬件
 - 执行控制器的操作（例如:控制器“告诉”数据通路，执行add指令，则数据通路就会将相应操作数传给加法器/运算器...）

≈≈处理器的四肢

- **控制器(CU:Control Unit)**是对数据通路要做什么操作进行调度的硬件
 - “告诉”数据通路需要执行什么操作？需要读内存吗？读哪个寄存器？需要写寄存器吗？写哪个寄存器？ ...

≈≈处理器的大脑

CPU的组成部分——示例

addi t0,x0,6

000000000110000000001010010011

数据通路

根据收到的控制器信息可知：

1. 操作数为0号寄存器的值和立即数“6”
2. 执行加法运算
—>选择0号寄存器的值和立即数作为操作数，并选择加法器的结果作为最终结果
3. 加法运算结果写入目的寄存器t0

控制器

决定：

- 要让数据通路执行什么操作？（人工查表...）
 - 加减乘除...、分支...
- 源操作数应该如何选择？
- 哪个寄存器？ t0(x5), x0
- 立即数是什么？
- 指令执行完，PC如何改变？

处理器设计步骤

- ① 分析指令系统，得出对数据通路的需求
- ② 为数据通路选择合适的组件
- ③ 根据指令需求连接组件建立数据通路
- ④ 分析每条指令的实现，以确定控制信号
- ⑤ 集成控制信号，形成完整的控制逻辑

支持简单指令集的RISC V处理器设计示例

设计一个能实现如下指令的RISC V CPU:

- 常用算术逻辑指令add、sub、addi、and、or等
- 存储器访问指令ld、sd等
- 条件分支指令beq(branch if equal)等
- 跳转指令jalr, jal
- auipc等指令
- 这个子集**不包含**所有的定点指令(例如移位、乘法、除法等), 也不包含任何浮点指令。

能实现其他指令的CPU设计与此类似。

RISC-V指令格式回顾及需求

- 简单源于规整

R型	func7	rs2	rs1	func3	rd	opcode
I型	imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12 ,10:5]	rs2	rs1	func3	imm[4:1, 11]	opcode
J型	imm[20 ,10:5]	imm[4:1, 11]	imm[19:15]	imm[14:12]	rd	opcode
U型	imm[31:25]	imm[24:20]	imm[19:15]	imm[14:12]	rd	opcode

RISC-V 部分指令需求-1

- R型指令: `op dst, src1, src2`
 - 需求: 算术逻辑运算器, 可读写的寄存器(可同时读2个)。
- I型指令: `opi dst, src, imm` # $R[dst] = R[src] + s\text{-}ext(imm)$
 - `load dst, offset(bAddrReg)` # $R[dst] = \underline{mem}[R[bAddrReg] + offset]$
 - `jalr rd, offset(rs1)` # $R[rd] = PC + 4$, $PC = R[rs1] + offset$
 - 新增需求: 立即数扩展、PC寄存器、存储器可读、加法器...
- S型指令: `store src, offset(bAddrReg)` # $\underline{mem}[R[bAddrReg] + offset] = R[src]$
 - 新增需求: 存储器可写、立即数生成器 (包括扩展等)

S型	imm[11:5]	rs2	rs1	func3	imm[4:0]	opcode
----	------------------	-----	-----	-------	-----------------	--------

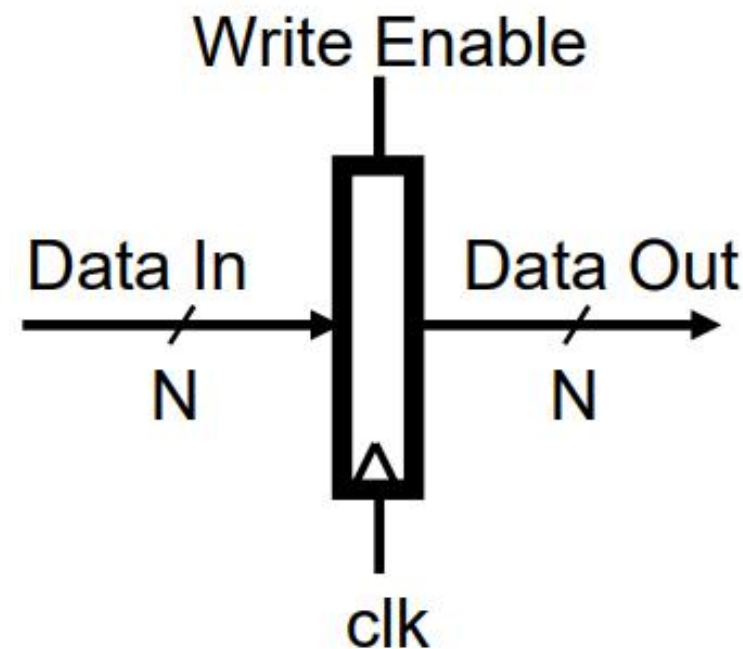
RISC-V 部分指令需求-2

- B型指令: bxx src1,src2, **label**
 - 新增需求: 多路选择器、更复杂的立即数生成器
- J型指令: jal rd, offset
 - 新增需求: 更复杂的立即数生成器
- U型指令: AUIPC将**长立即数**加到PC并写入目的寄存器
 - 新增需求: 更复杂的立即数生成器

B型	imm[12 ,10:5]	rs2	rs1	func3	imm[4:1, 11]	opcode
J型	imm[20 ,10:5]	imm[4:1, 11]	imm[19:15]	imm[14:12]	rd	opcode
U型	imm[31 :25]	imm[24:20]	imm[19:15]	imm[14:12]	rd	opcode

本设计实例对数据通路(组件)的需求汇总

- 算术逻辑单元 (ALU)
 - 运算类型：加、减、或、比较等各种运算
- 立即数生成器
 - 零扩展、符号扩展、字段拼接等
- 程序计数器 (PC)
- 存储组件：寄存器堆(RF: Register File), 存储器(Memory)



第4章 处理器设计

- 处理器设计的需求分析
- **RISC-V数据通路的组件选择**
- RISC-V部分指令的数据通路设计
- RISC-V控制器

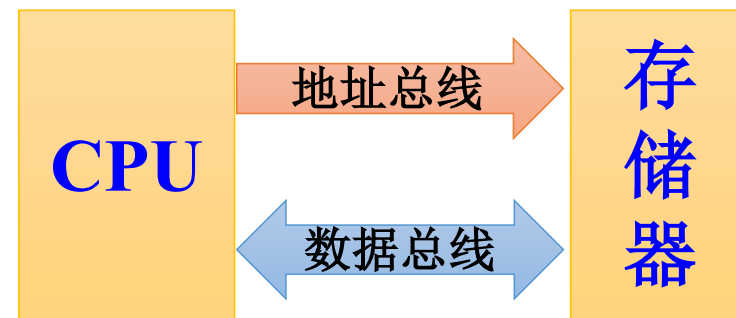
数据通路组件分类

- 组合逻辑单元
 - 处理数据的单元
 - 没有存储功能
 - 输出仅依赖于输入：输入相同，则输出相同
 - 如基本逻辑门、ALU等
- 时序逻辑单元（状态单元）
 - 存储数据的单元
 - 有内部存储功能，即包含“状态”
 - 下一状态取决于输入和当前的状态
 - 如指令存储器、数据存储器以及寄存器

冯诺依曼架构vs 哈佛架构

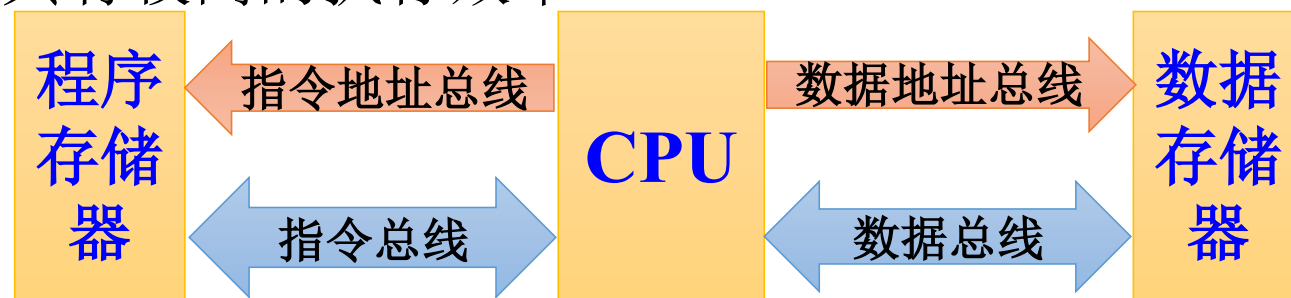
- 冯·诺依曼架构（**普林斯顿架构**，以X86为代表）

- 程序空间和数据空间一体，数据和程序采用同一数据总线和地址总线。
- 指令和数据的**宽度相同**。
- 指令和数据不能同时进行操作，只能顺序执行。



- 哈佛架构（以DSP和ARM为代表）

- 存储器分为数据存储器 and 程序存储器，总线分为程序存储器的数据总线和地址总线以及数据存储器的数据总线和地址总线。
- 可同时对数据和程序进行操作，具有较高的执行效率。
- 指令和数据可以有**不同的宽度**。

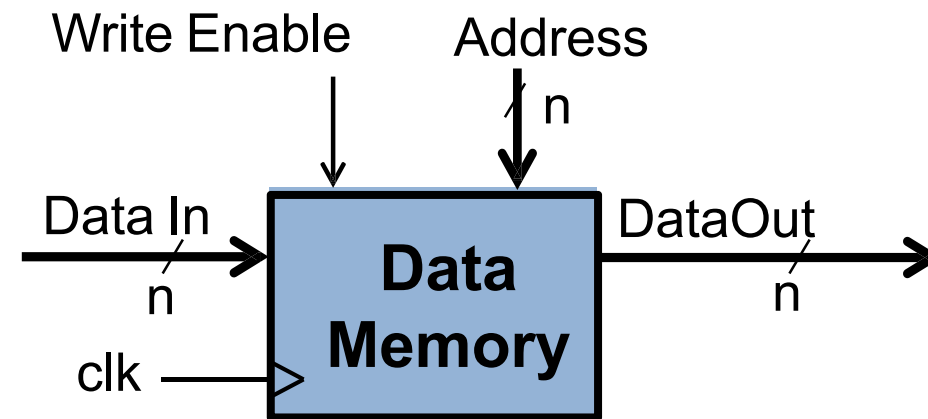


RISC V数据通路中主要状态单元——存储器

- 将指令和数据保存在**字节寻址**的存储空间中
- 指令存储器(IM: Instruction Memory)和数据存储器(DM: Data Memory)**分开**（RISC CPU采用的是**哈佛**架构）
- 从指令存储器**读**（取）指令，在数据存储器中**读写**数据
- 指令和数据的位宽可以不同

数据存储 (DM: Data Memory)

- 组合逻辑 vs 时序逻辑
- 数据接口信号(总线)
 - Data In: 数据输入
 - Data Out: 数据输出
- 读写控制



- **Address:** 指定一个存储单元，将其内容送到数据输出端
- **WriteEnable:** 写使能。在时钟信号（**clk**）的上升沿，如果该信号有效，将输入数据存入地址信号指定的存储单元

注：存储器的读操作不受时钟控制(Ideal Data Memory)

指令存储器 (IM:Instruction Memory)

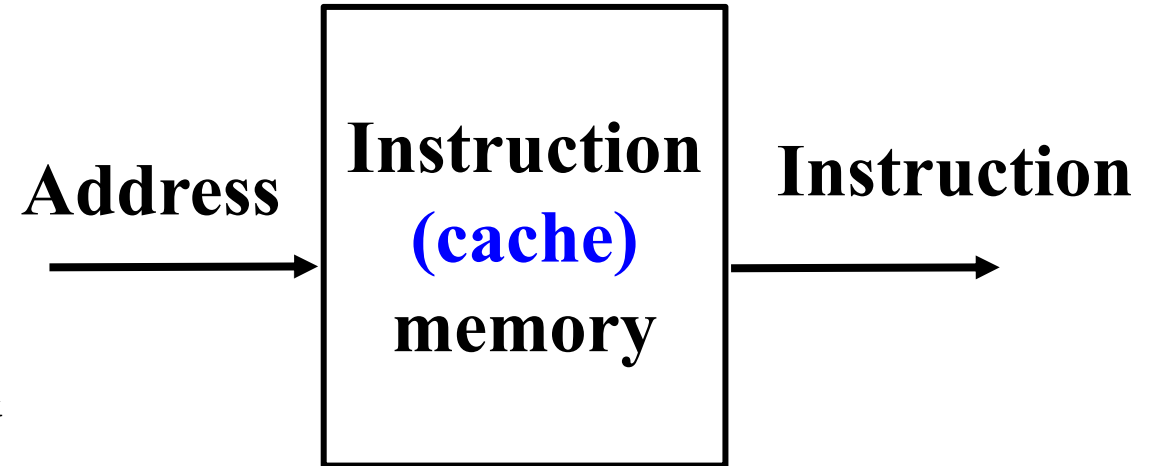
- 输入总线: Instruction Address

- 输出总线: Instruction

- 数据通路中不会写指令存储器

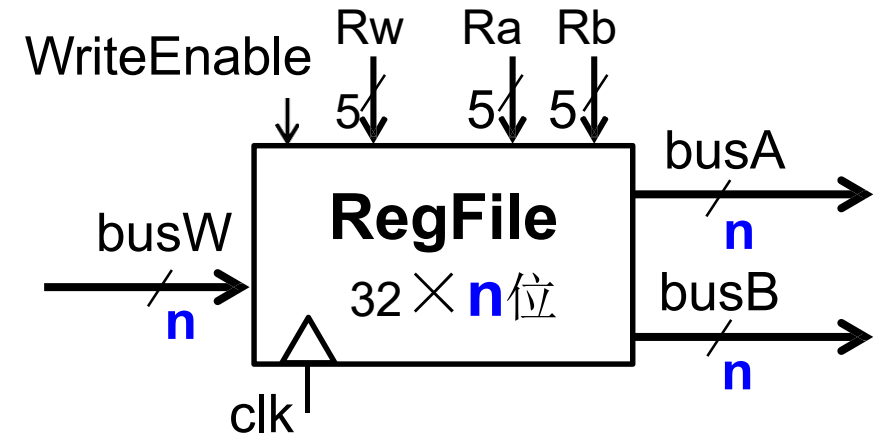
- Ideal Instruction Memory

- 读操作时, 指令存储器可以看做是组合逻辑电路。



数据通路中主要状态单元——寄存器堆(RF)

- (RegisterFile)内部构成：32个寄存器
- 数据接口信号
 - busA, busB: 两个数据输出
 - busW: 一个数据输入
- 读写控制

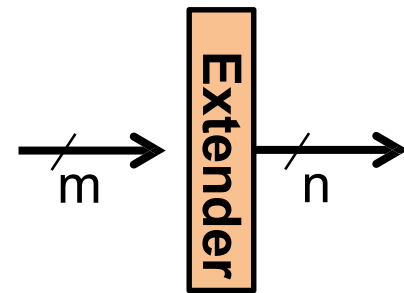
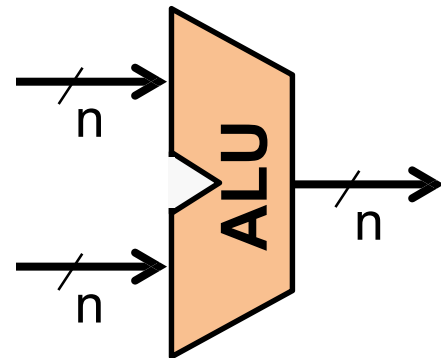


- Ra/Rb(5位): 将对应编号的寄存器的内容放到busA/busB(读)
- Rw(5位): 在时钟信号(**clk**)的上升沿, 如果写使能信号(**WriteEnable**)有效, 将busW的内容存入Rw号寄存器。

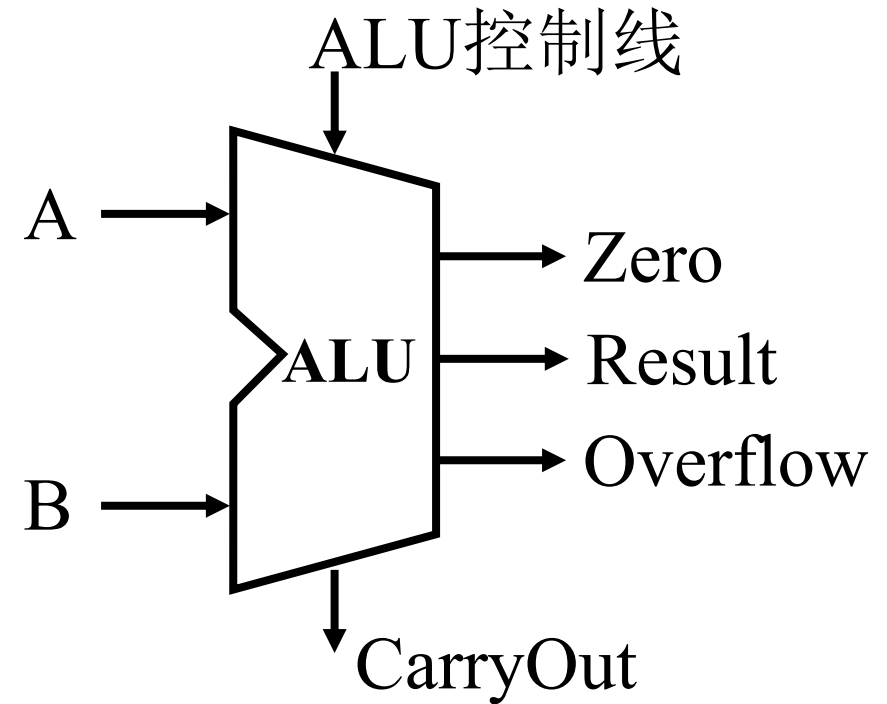
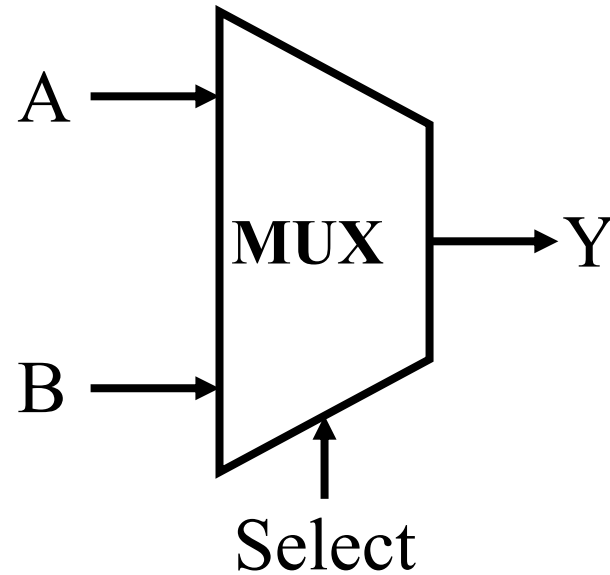
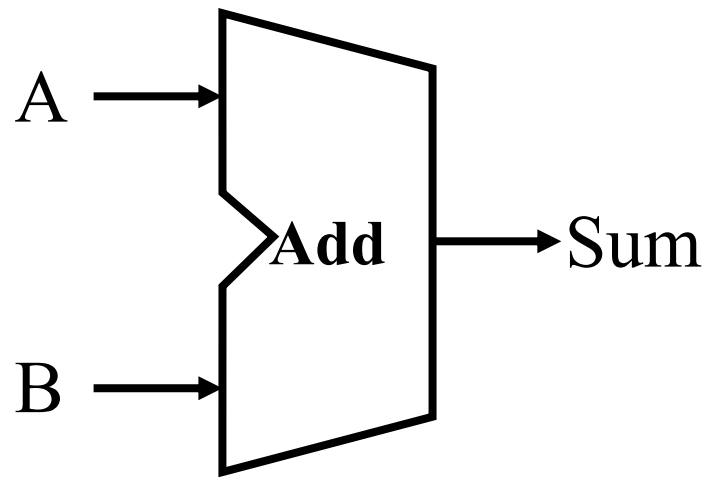
注: 寄存器堆的读操作不受时钟控制, 组合逻辑电路。

RISC-V指令系统的数据通路需求

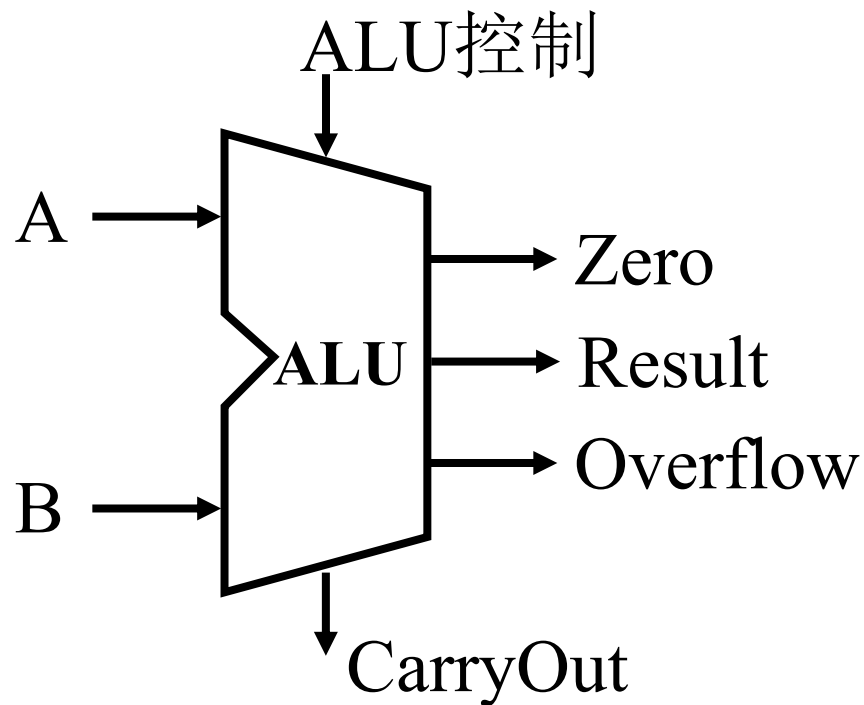
- 算术逻辑单元（Arithmetic Logic Unit）
 - 运算类型：加、减、或、比较、与、或等
 - 操作数：来自寄存器或扩展后的立即数
- 立即数扩展(生成)部件
 - 将一个不足32/64位的立即数扩展为32/64位
 - 扩展方式：零扩展、符号扩展、拼接
- 程序计数器（Program Counter）
 - 一个32/64位的寄存器
 - 两种加法：加4 或 加一个立即数



数据通路模块——常用组合逻辑单元



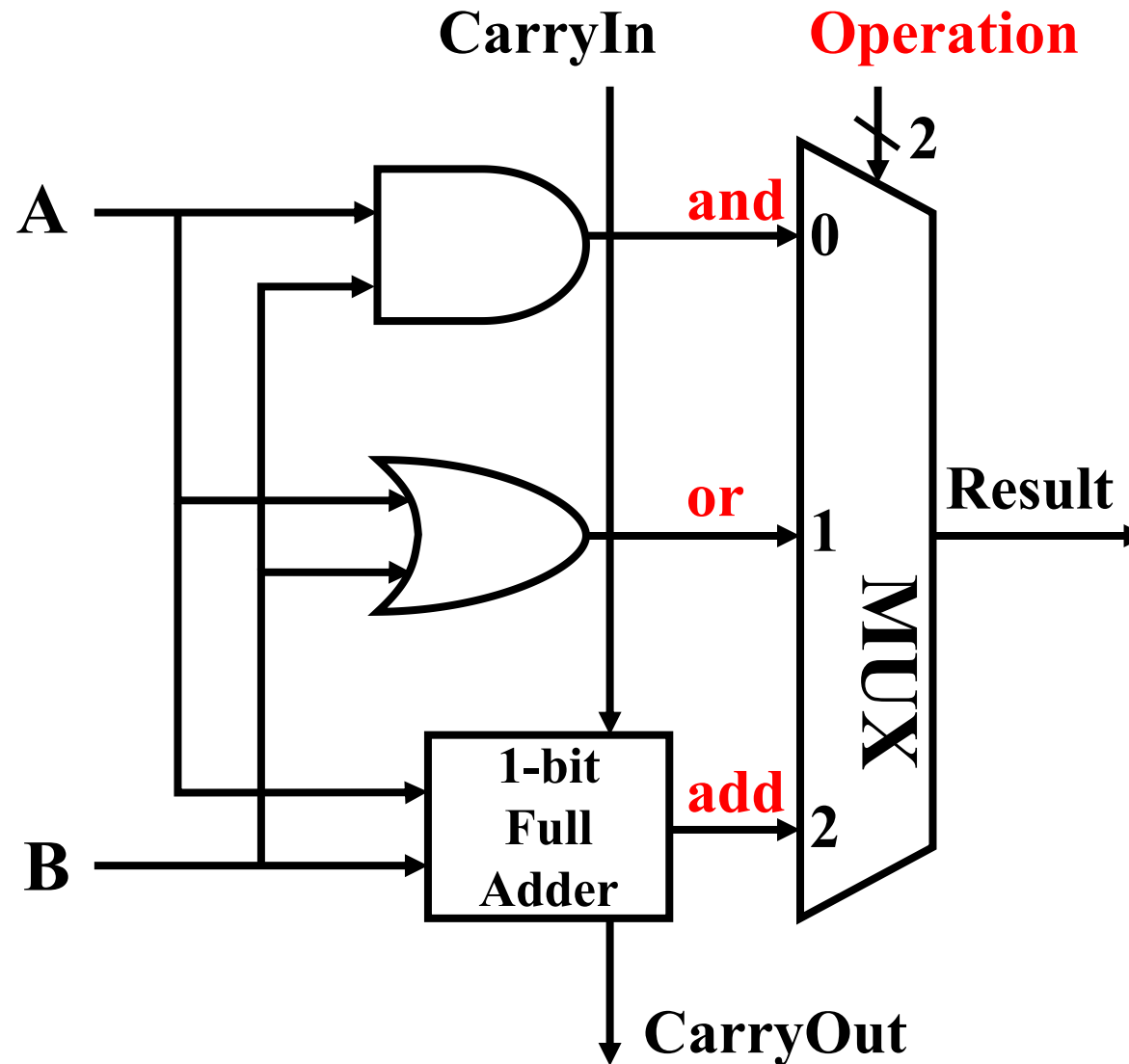
ALU功能需求及设计思路



ALU控制	操作
0000	与(and)
0001	或(or)
0010	加(add)
0110	减(sub)
0111	小于则置位(slt)
1100	或非(nor)

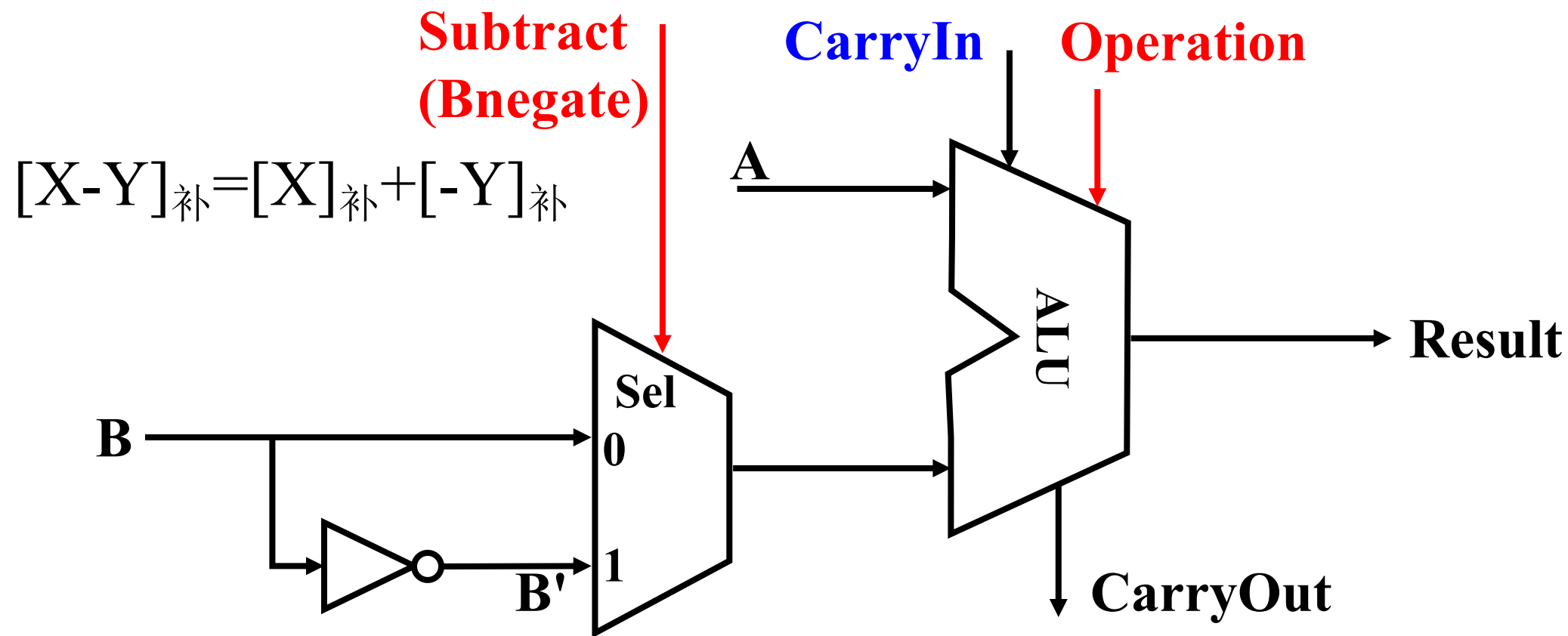
- 从简单开始：然后再进行扩展
- 分而治之：从1位ALU设计开始
- 利用已知的元件或者组件，组合起来实现复杂功能

ALU——1位简单运算



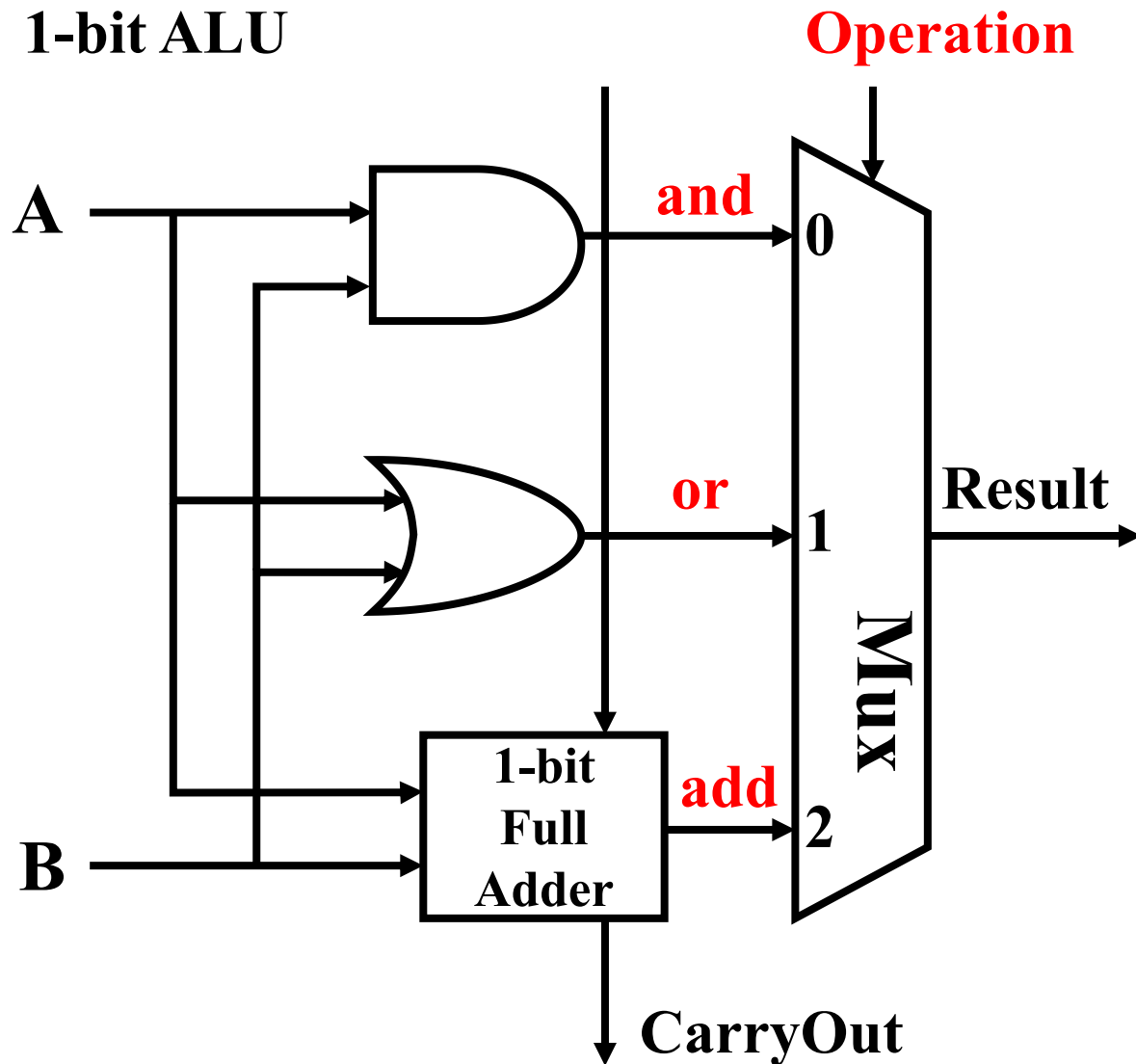
ALU——如何实现1位减法运算

$$A - B = A + B' + 1$$

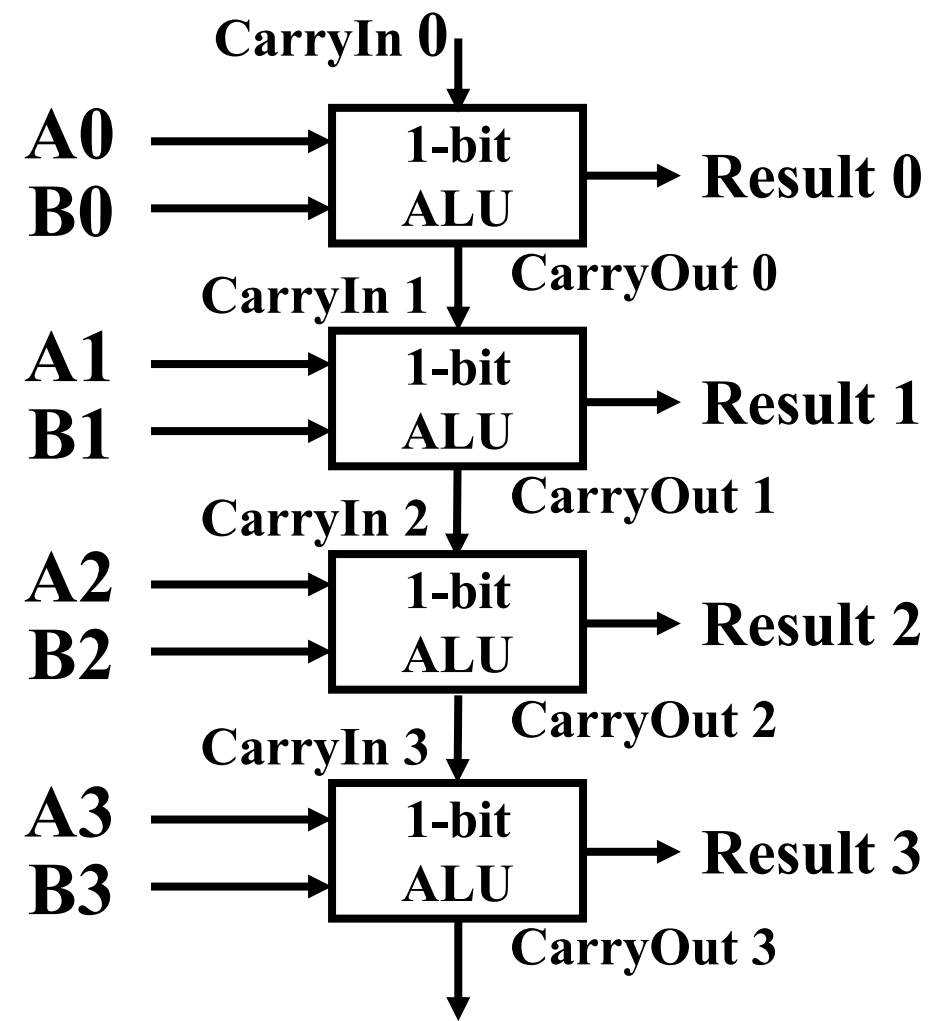


1位到多位ALU

1-bit ALU

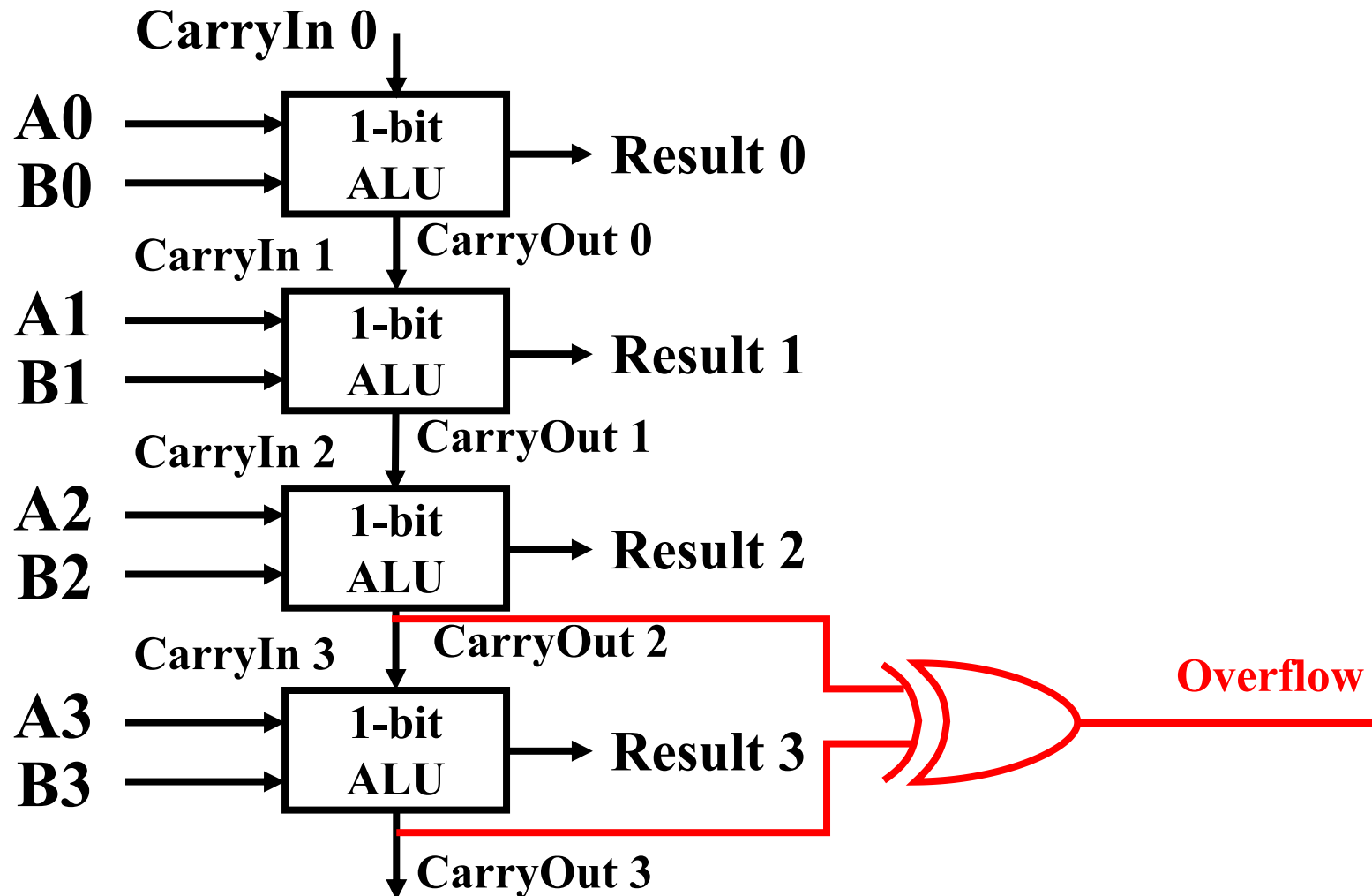


4-bit ALU

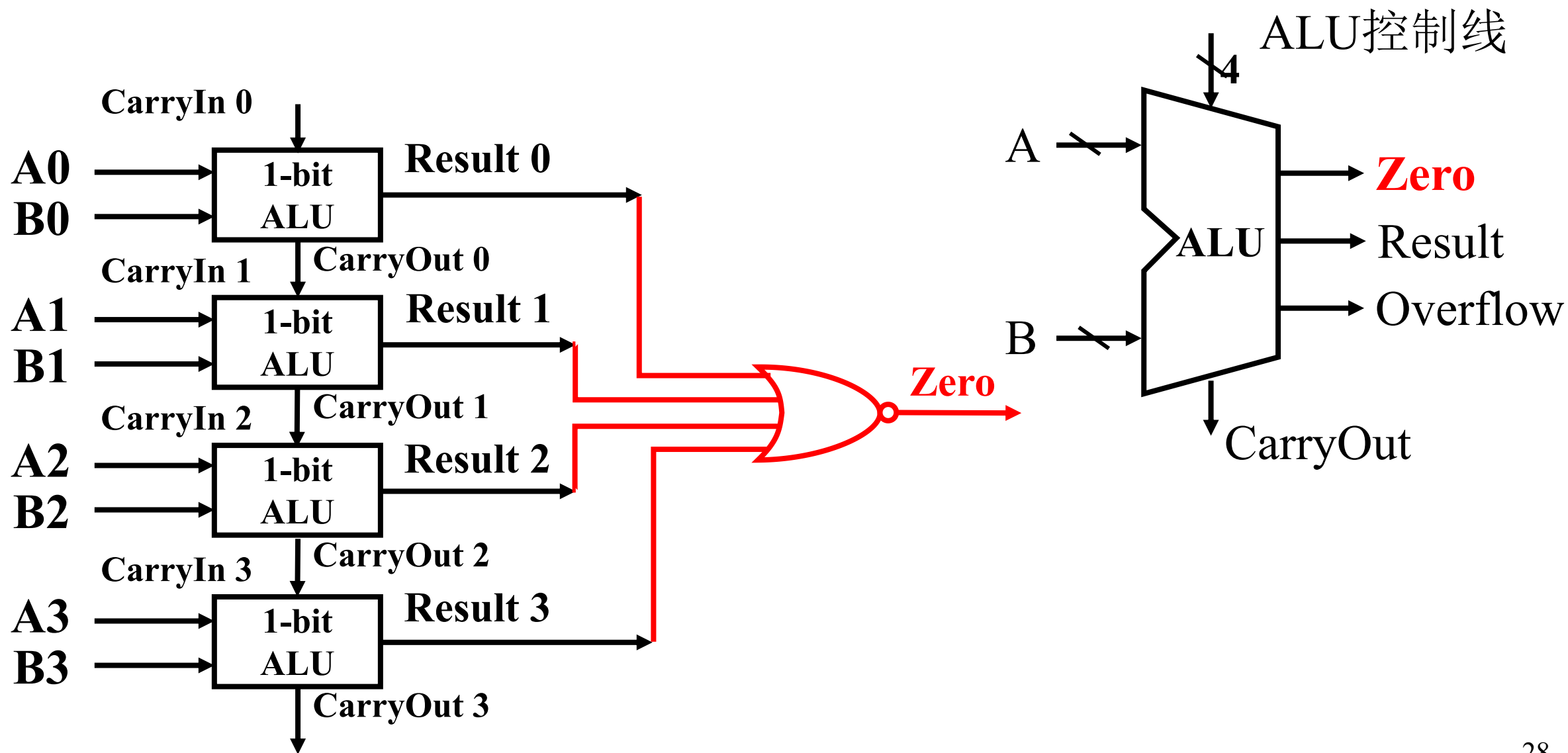


ALU——溢出检测逻辑

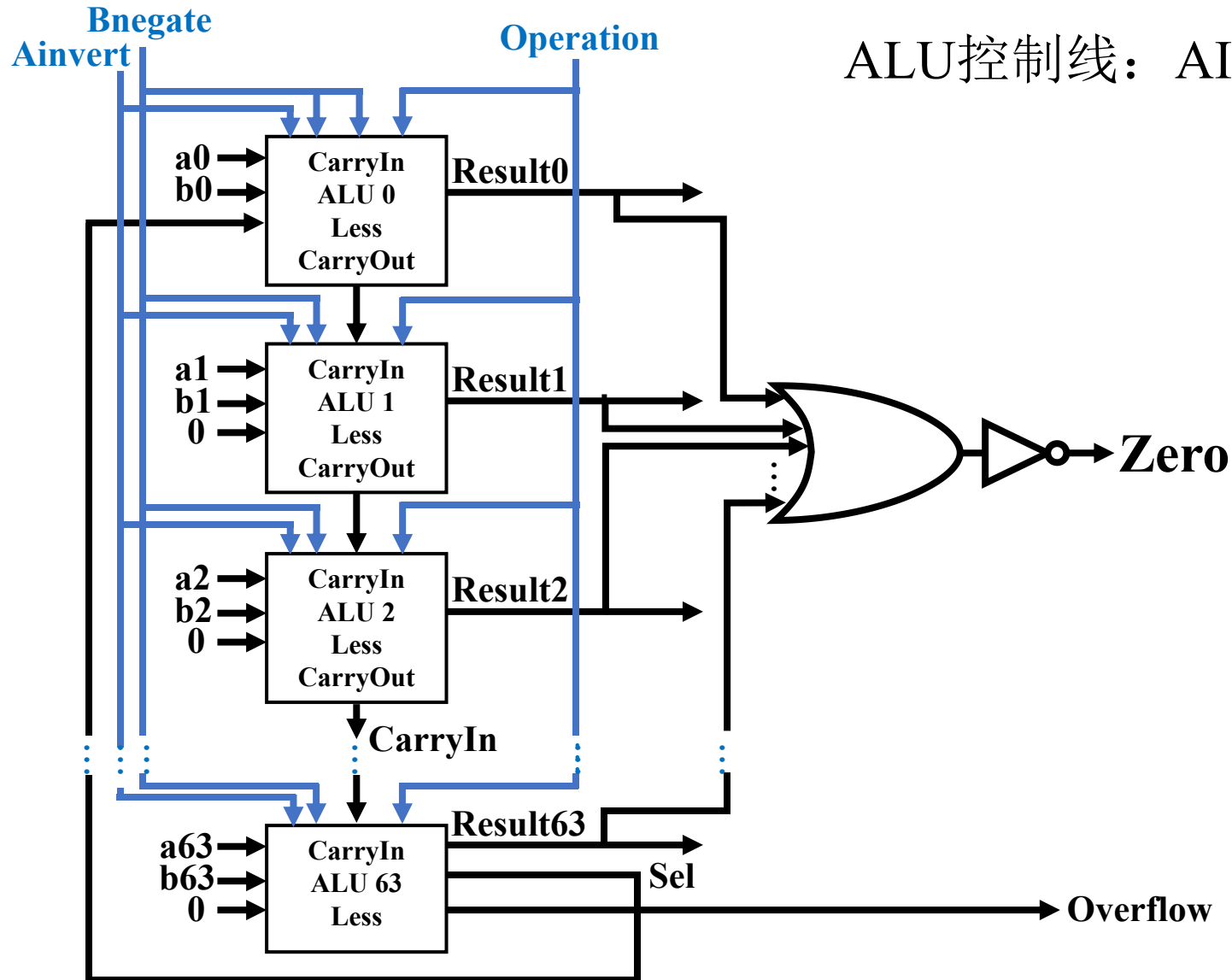
$$\text{Overflow} = \text{CarryIn}[N-1] \text{ xor } \text{CarryOut}[N-1]$$



ALU——判零逻辑



64bit ALU



ALU控制线: AInvert, Bnegate, Operation

ALU控制 (ALUControl)	操作
0000	与(and)
0001	或(or)
0010	加(add)
0110	减(sub)
0111	小于则置位(slt)
1100	或非(nor)

第4章 处理器设计

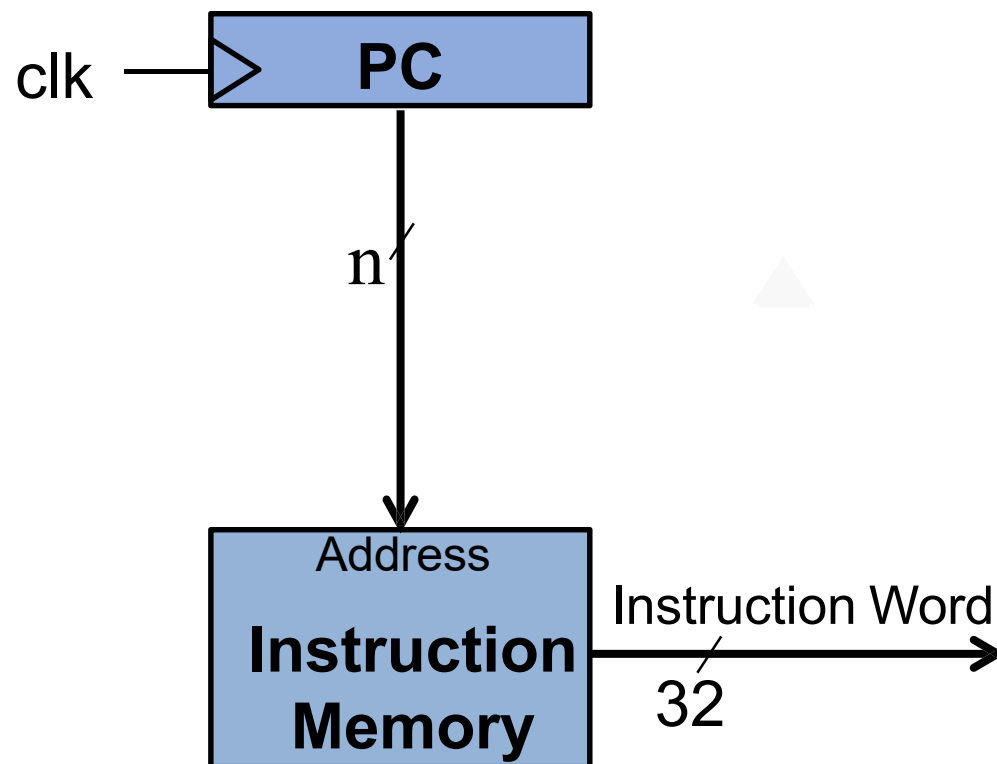
- 处理器设计的需求分析
- RISC-V数据通路的组件选择
- **RISC-V部分指令的数据通路设计**
- RISC-V控制器

处理器的设计步骤

- ① 分析指令系统，得出对数据通路的需求
- ② 为数据通路选择合适的组件
- ③ 根据指令需求连接组件建立数据通路
- ④ 分析每条指令的实现，以确定控制信号
- ⑤ 集成控制信号，形成完整的控制逻辑

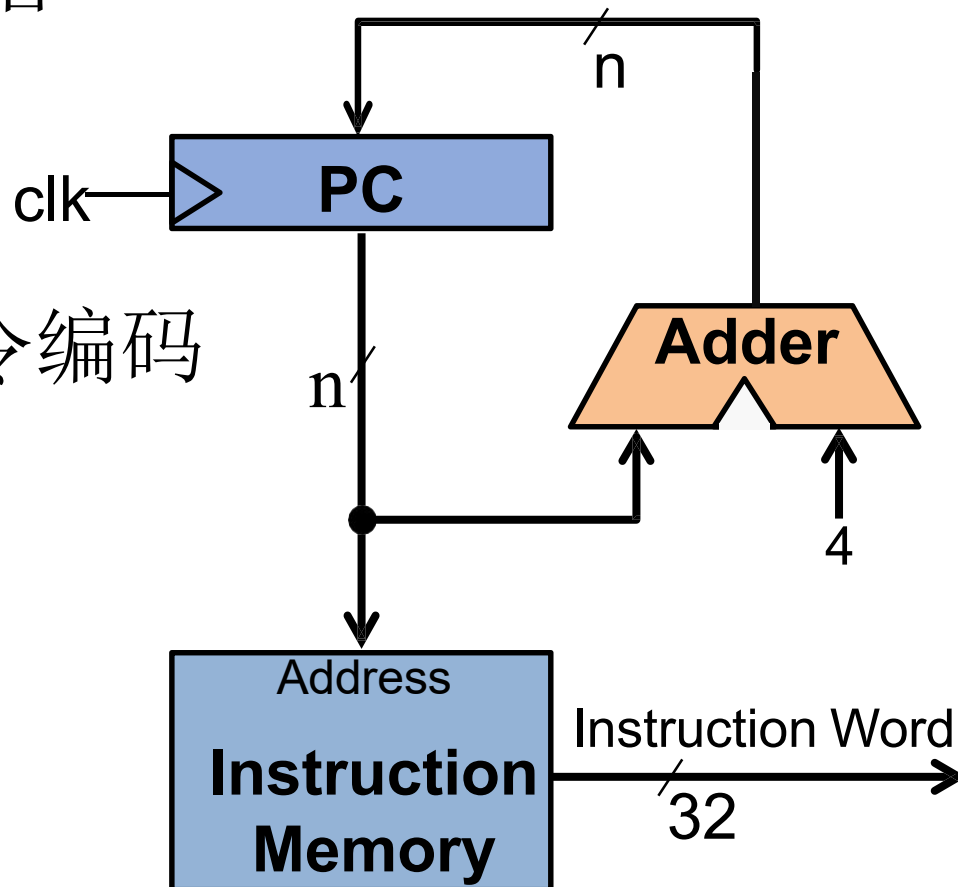
所有指令的共同需求——取指令

- 程序计数器（PC）的内容是当前指令的地址
- 用PC的内容作为地址，访问指令存储器获得指令编码



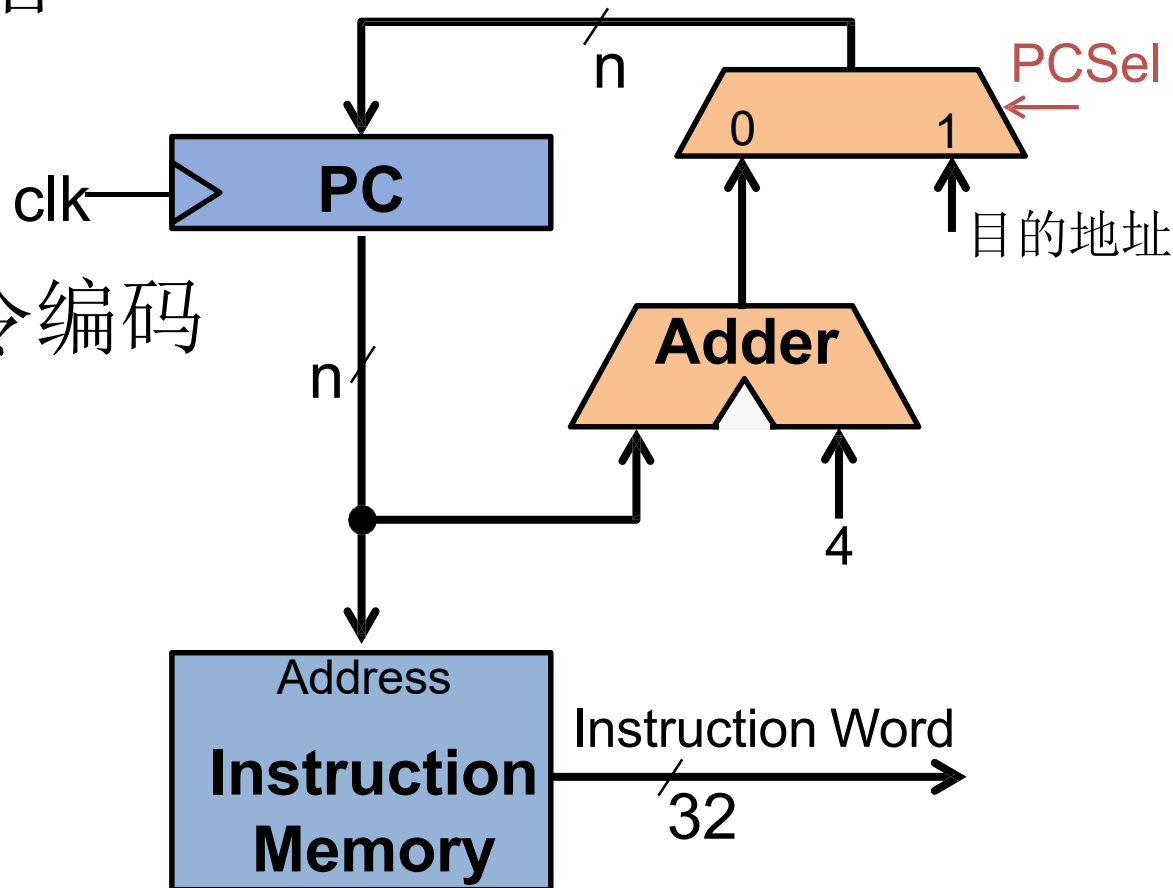
所有指令的共同需求——更新PC

- 取指令
 - 程序计数器（PC）的内容是当前指令的地址
 - 用PC的内容作为地址，访问指令存储器获得指令编码
- 顺序执行时更新PC
 - $PC \leftarrow PC + 4$
- 发生分支时更新
 - $PC \leftarrow \text{分支目标地址}$



所有指令的共同需求——更新PC（分支）

- 取指令
 - 程序计数器（PC）的内容是当前指令的地址
 - 用PC的内容作为地址，访问指令存储器获得指令编码
- 顺序执行时更新PC
 - $PC \leftarrow PC + 4$
- 发生分支时更新
 - $PC \leftarrow$ 分支目标地址



所有指令的共同需求

- 取指令

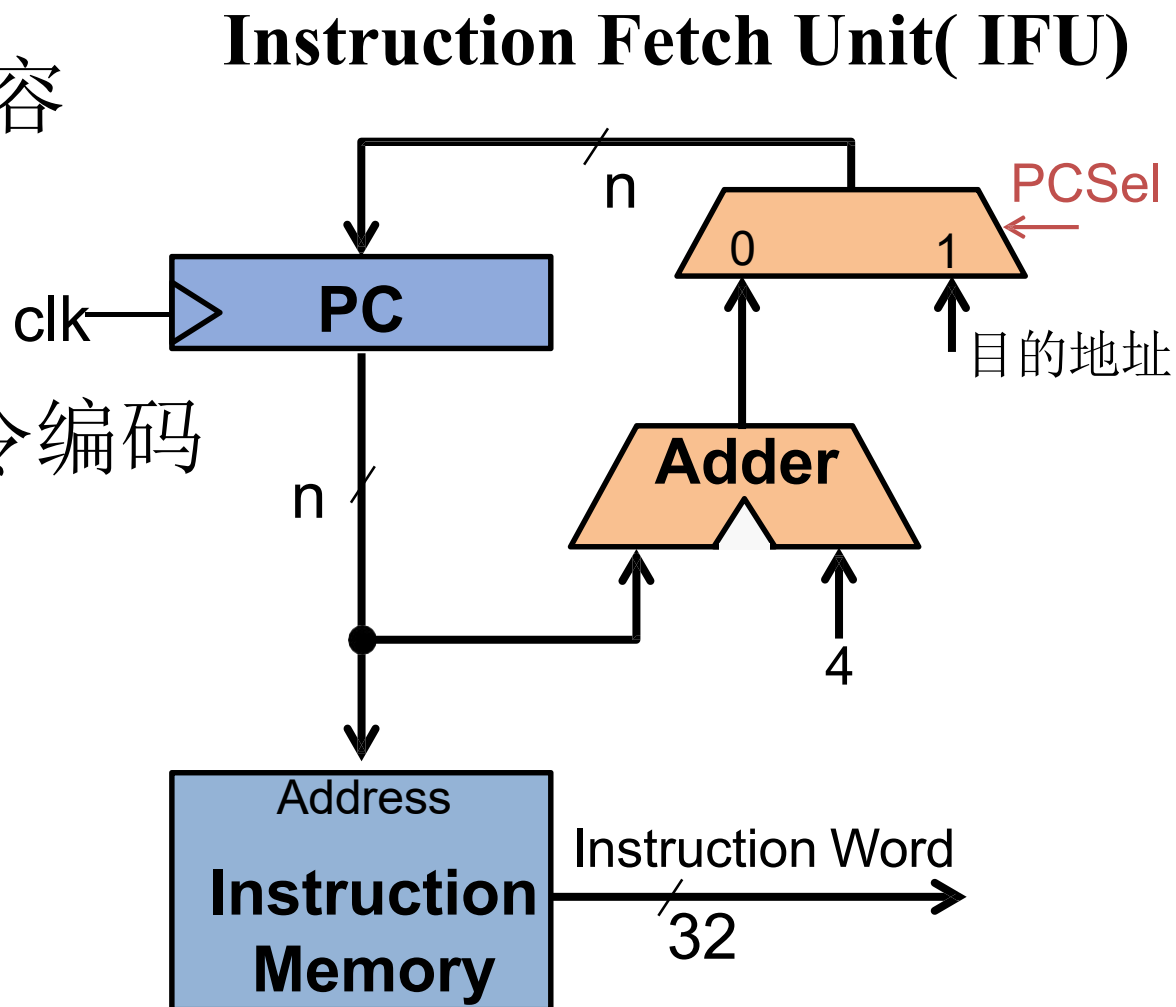
- 程序计数器（PC）的内容是当前指令的地址
- 用PC的内容作为地址，访问指令存储器获得指令编码

- 顺序执行时更新PC

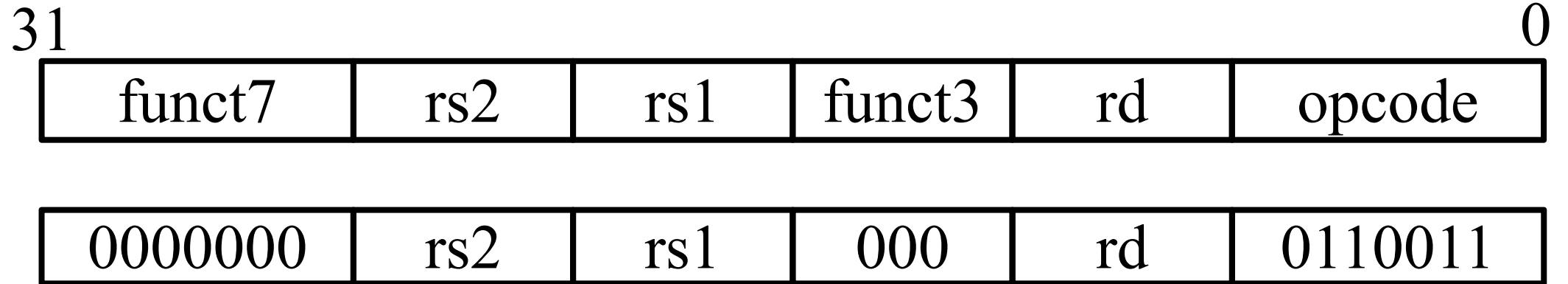
- $PC \leftarrow PC + 4$

- 发生分支时更新

- $PC \leftarrow$ 分支目标地址



实现add指令

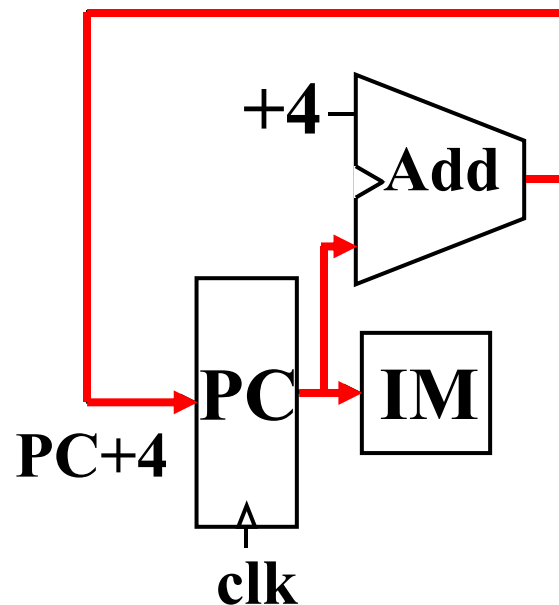


add rd, rs1, rs2

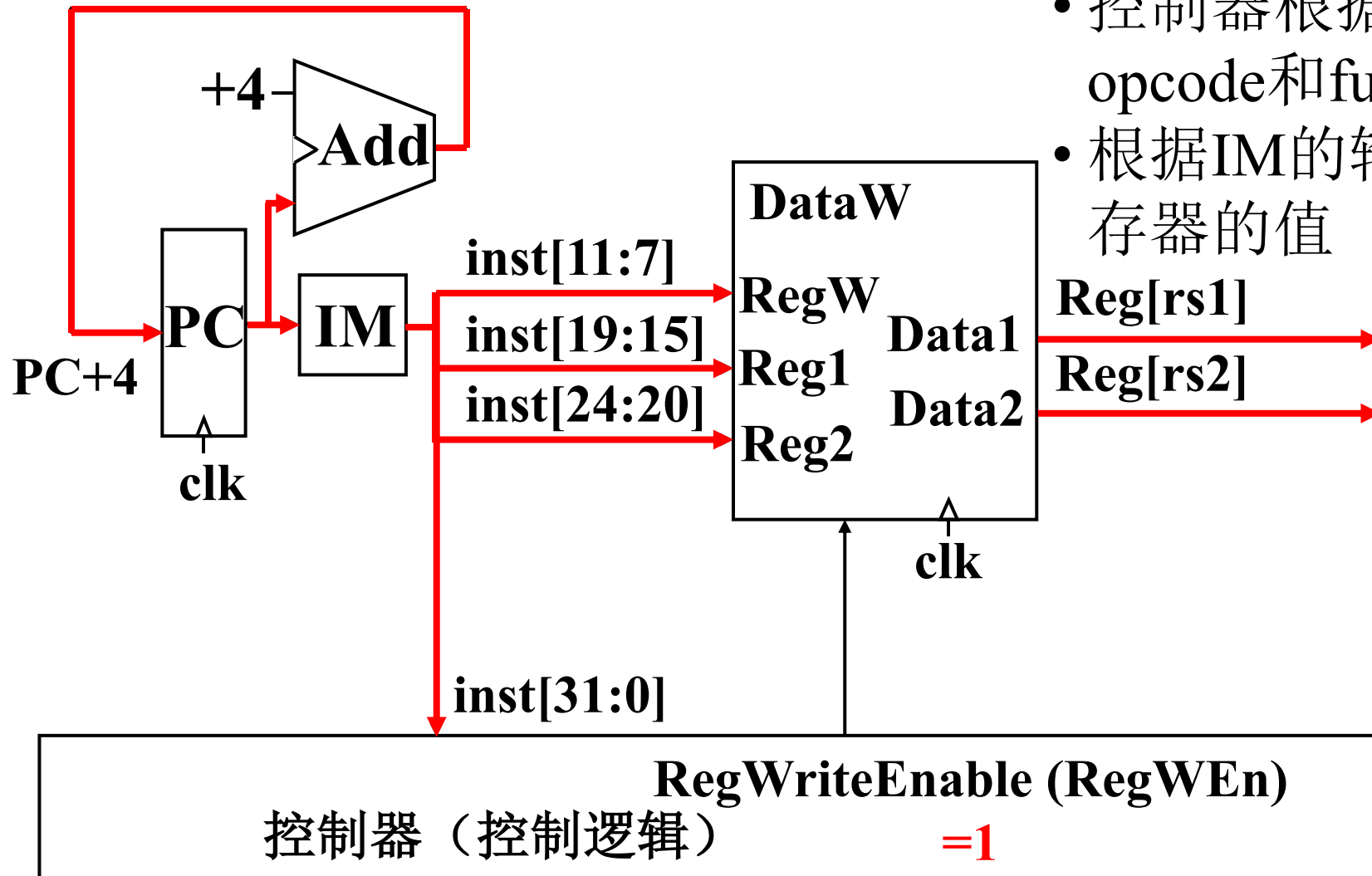
- 指令对机器状态进行了两次更改:
 - $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] + \text{Reg}[\text{rs2}]$
 - $\text{PC} = \text{PC} + 4$

add指令的数据通路——取指

- 根据PC从IM中读出指令
- 通过加法器计算下一个PC值($PC+4$)

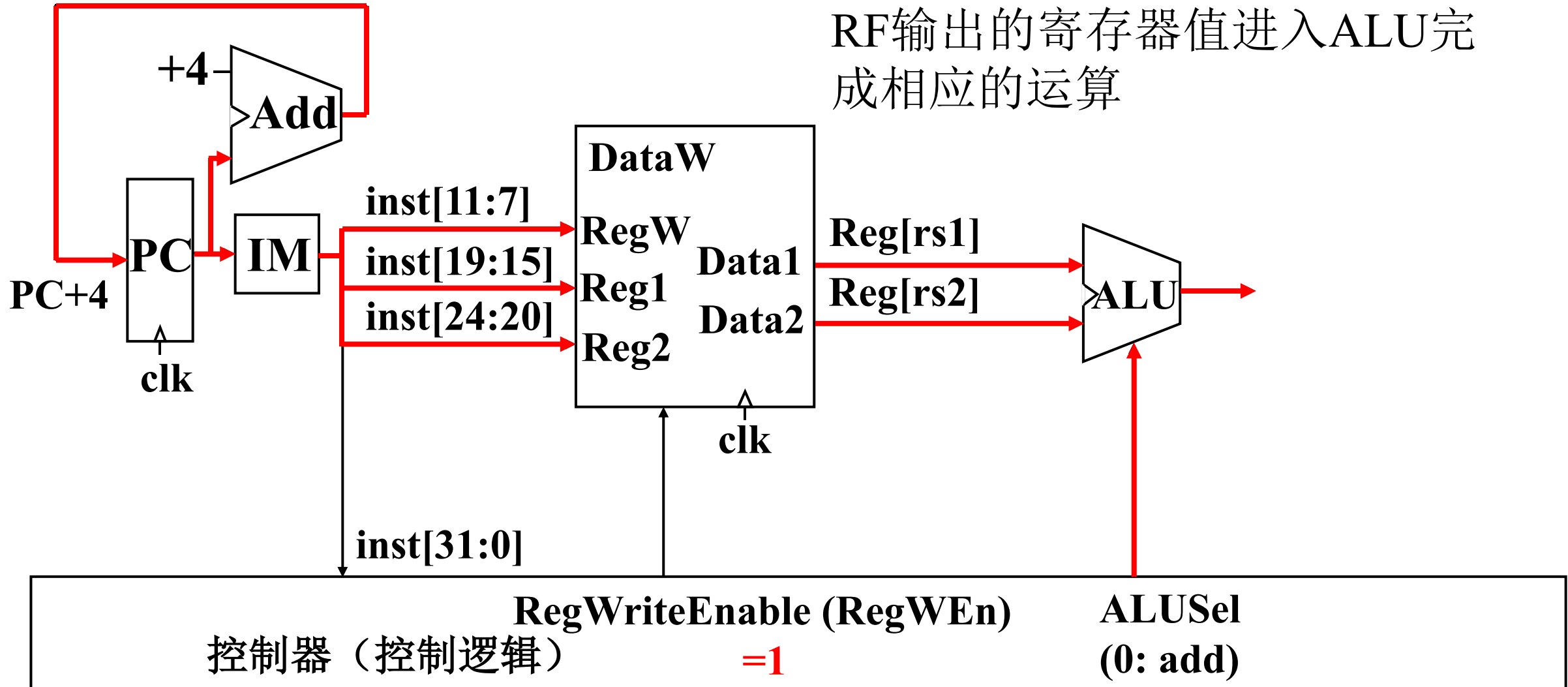


add指令的数据通路——译码

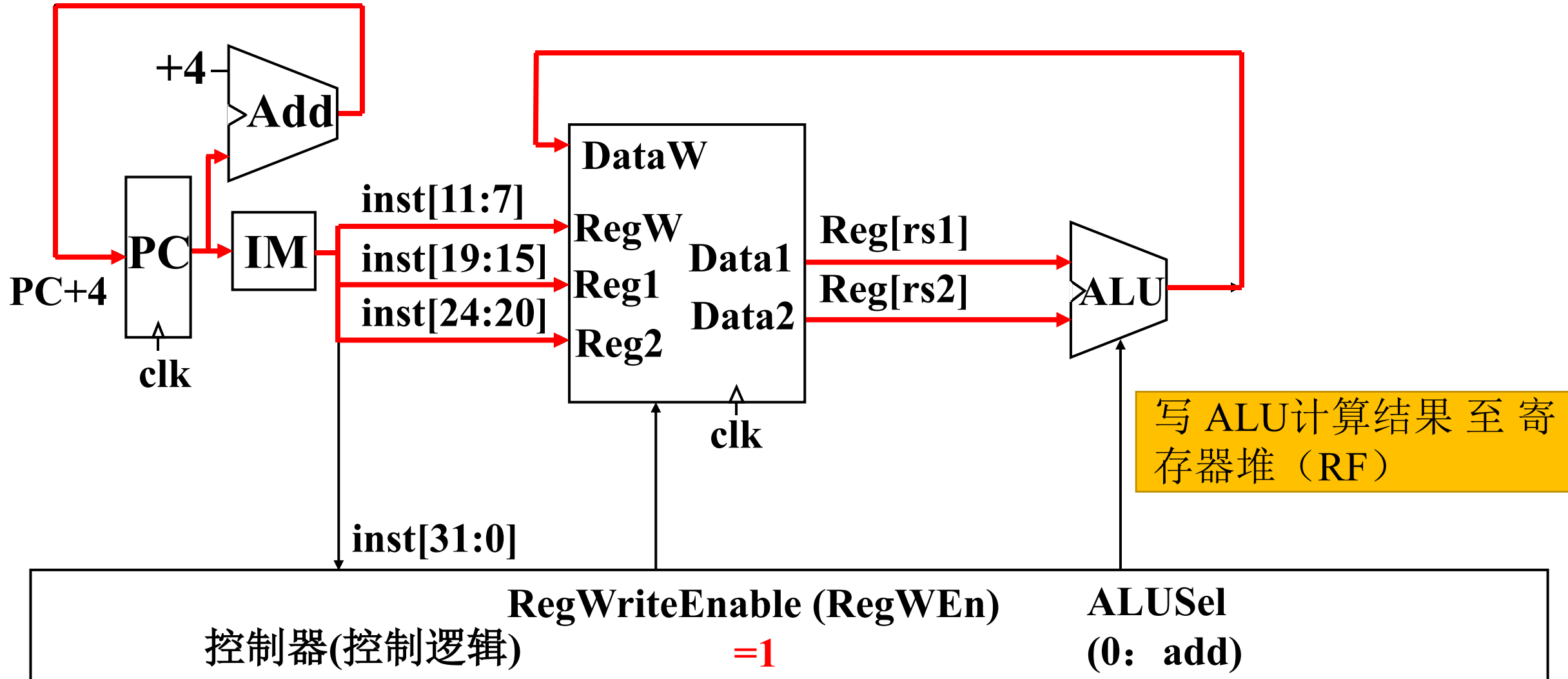


- 控制器根据IM的输出分析指令的opcode和funct字段，确定控制信号
- 根据IM的输出读出RF中相应2个寄存器的值

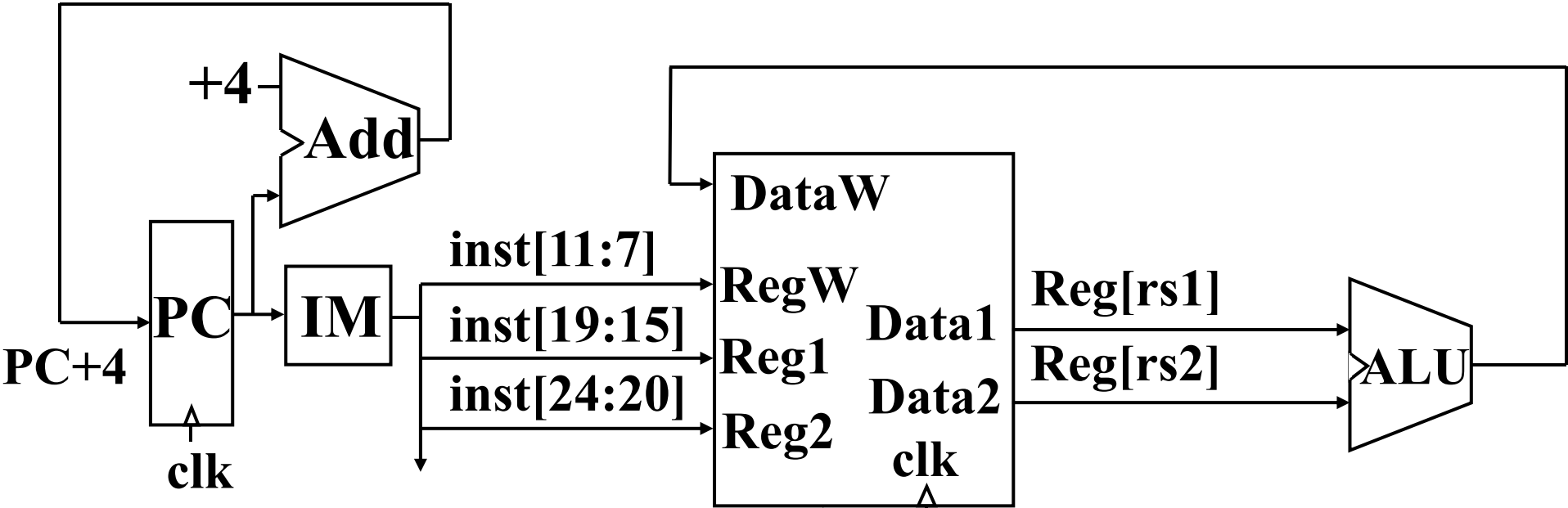
add指令的数据通路——执行



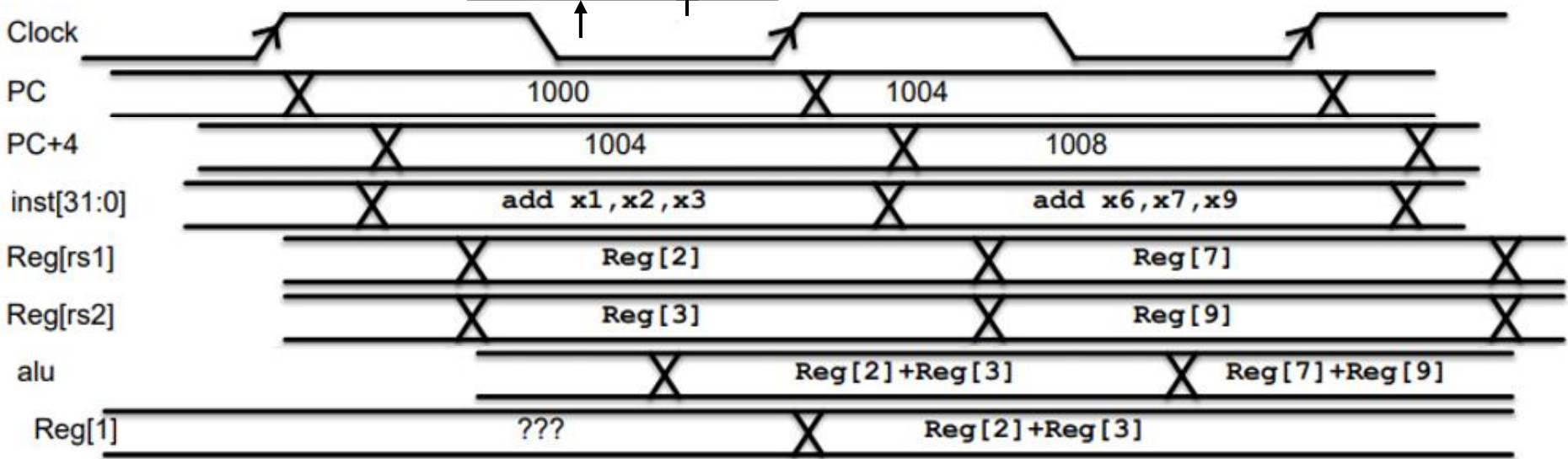
add指令的数据通路——写回寄存器



add指令的时序图



PC时延
指令存储器时延
寄存器堆时延
ALU时延



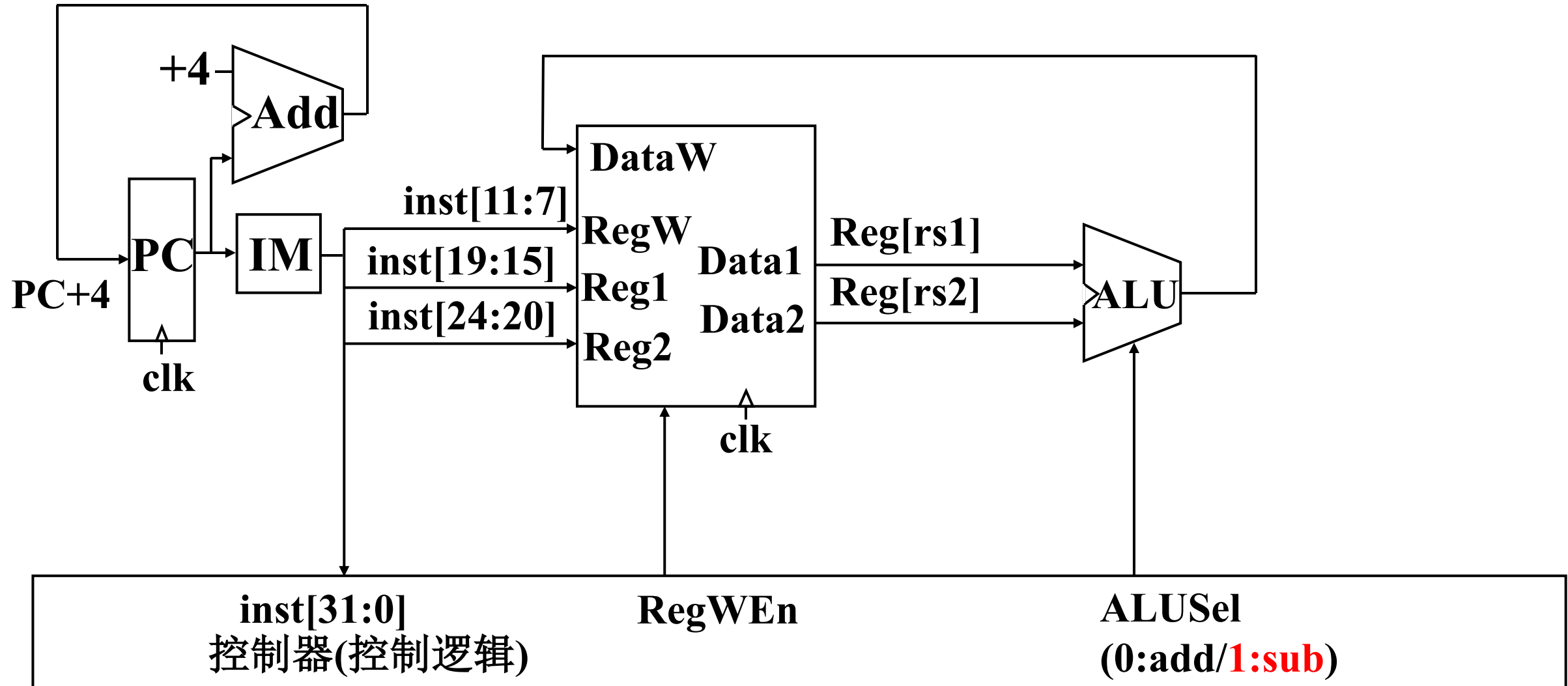
实现sub指令

31						0	
	0000000	rs2	rs1	funct3	rd	0110011	add
	0 1 00000	rs2	rs1	000	rd	0110011	sub

sub rd, rs1, rs2

- 几乎与加法相同
- **inst[30]**在加法和减法之间进行选择

sub指令的数据通路



实现R型部分指令

0000000	rs2	rs1	000	rd	0110011	add
0 1 00000	rs2	rs1	000	rd	0110011	sub
0000000	rs2	rs1	001	rd	0110011	sll
0000000	rs2	rs1	010	rd	0110011	slt
0000000	rs2	rs1	011	rd	0110011	sltu
0000000	rs2	rs1	100	rd	0110011	xor
0000000	rs2	rs1	101	rd	0110011	srl
0 1 00000	rs2	rs1	101	rd	0110011	sra
0000000	rs2	rs1	110	rd	0110011	or
0000000	rs2	rs1	111	rd	0110011	and

opcode、funct3和funct7字段确定ALU的不同功能

回顾addi指令

addi rd, rs1, imm

addi x15, x1, -50

31

0

imm[11:5]	imm[4:0]	rs1	func3	rd	opcode
1111110	01110	00001	000	01111	0010011

imm= -50

rs1=1

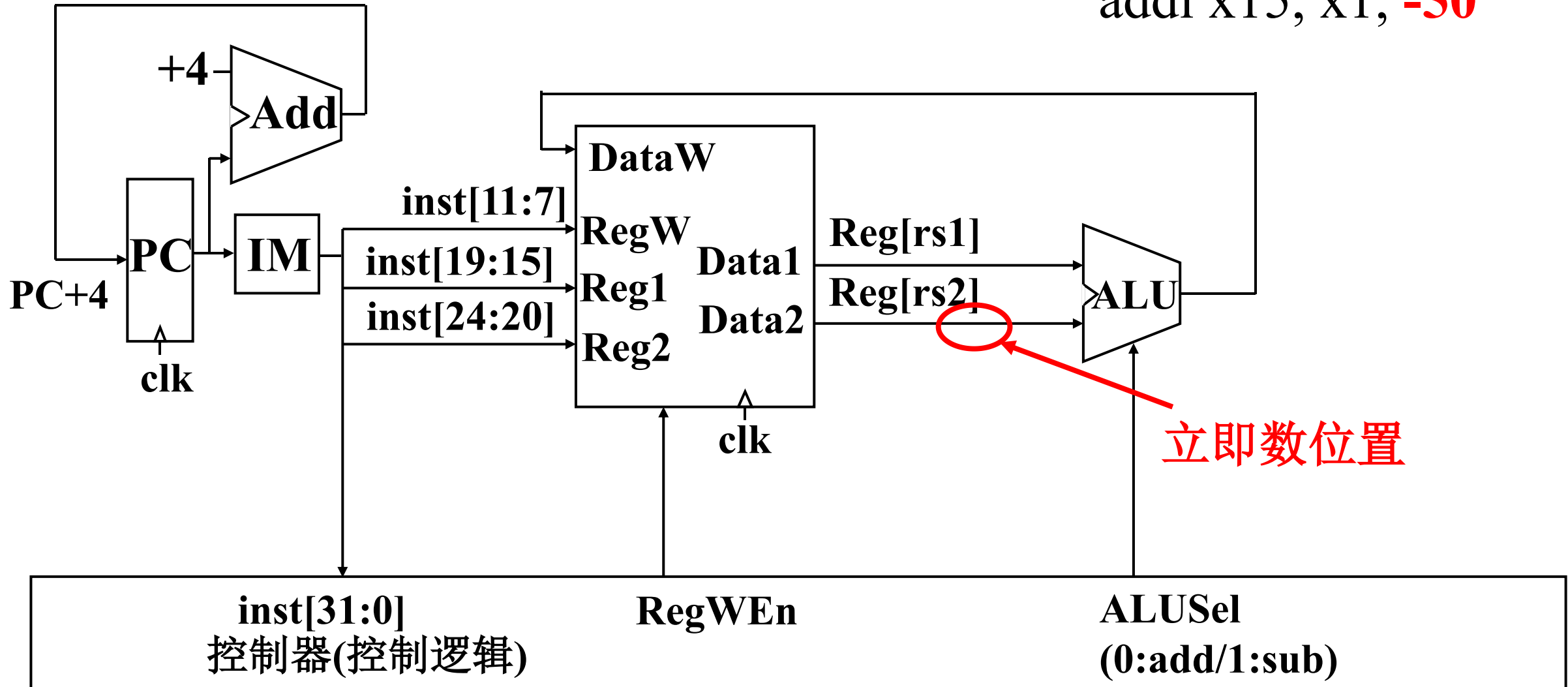
add

rd=15

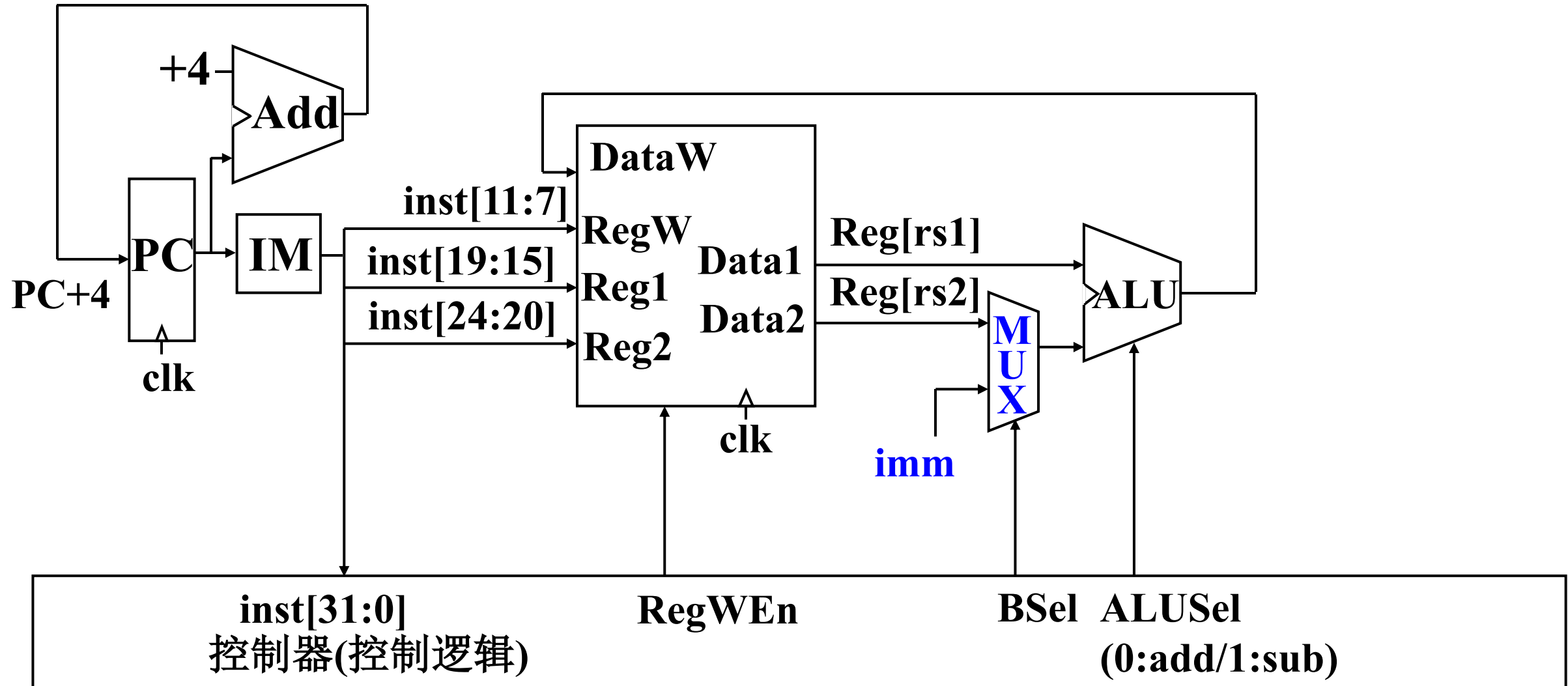
OP-Imm

add指令数据通路—>addi?

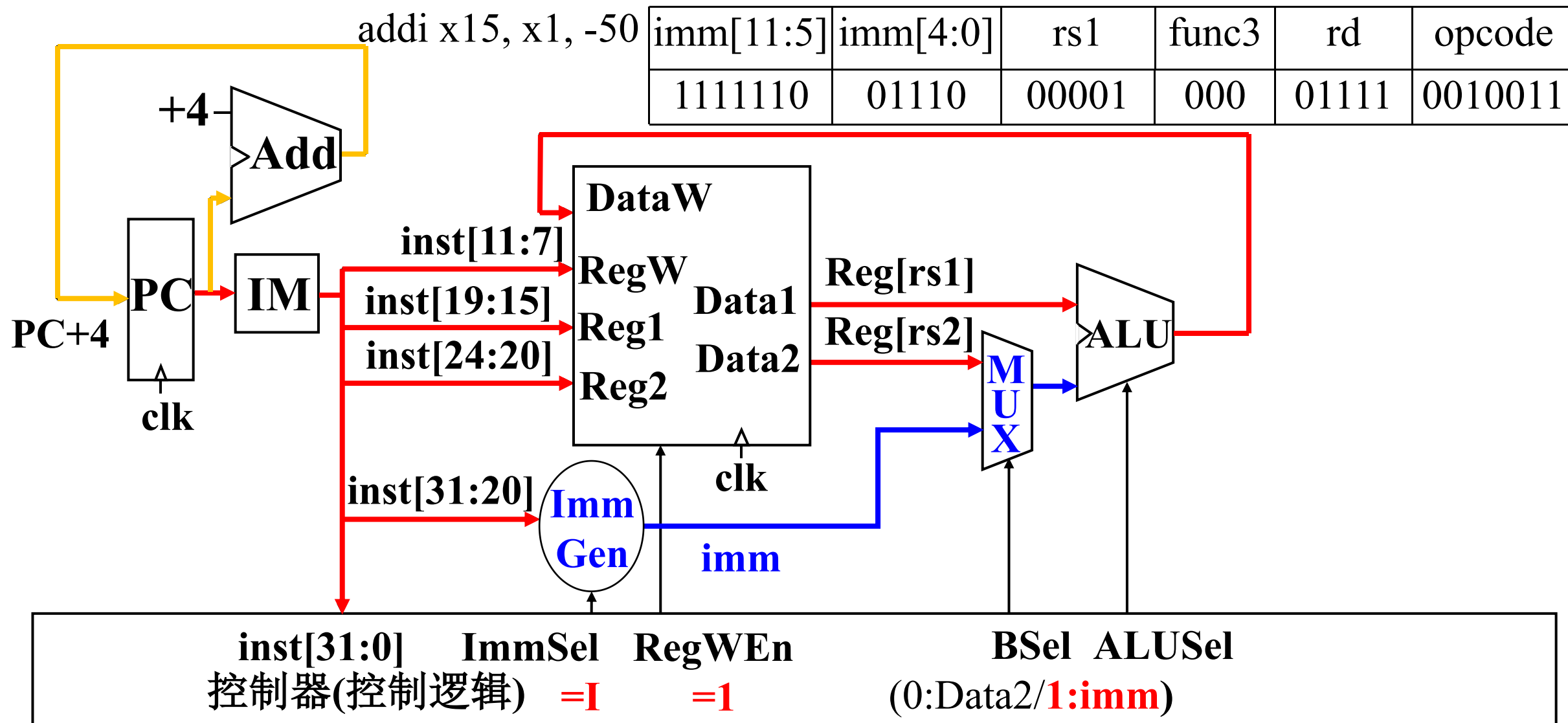
addi x15, x1, **-50**



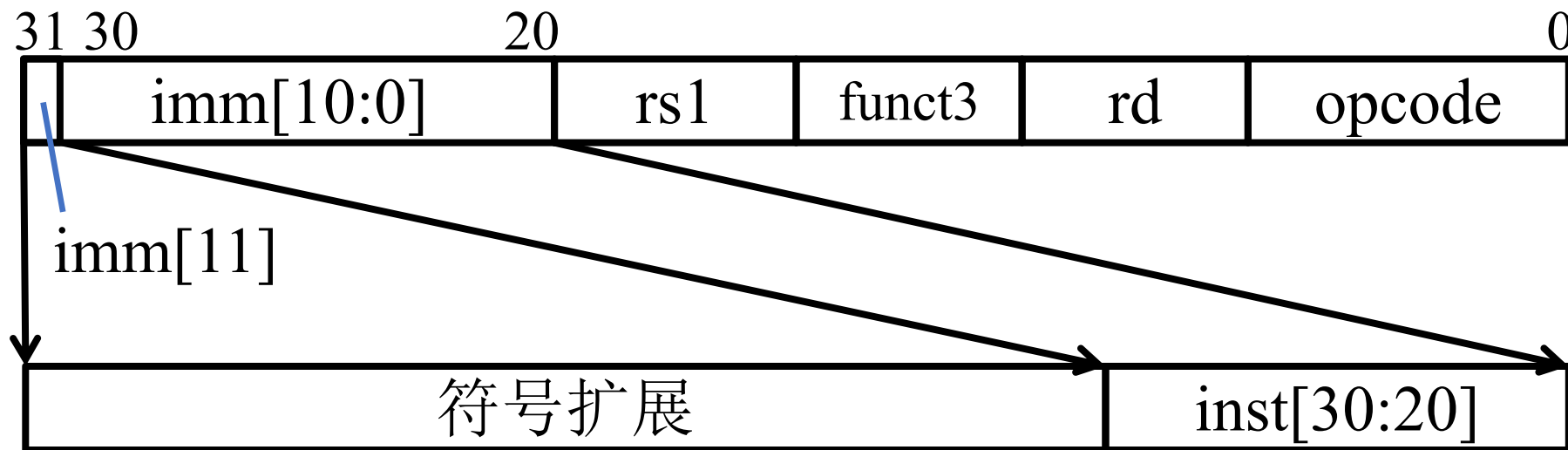
addi指令的数据通路



addi指令的数据通路



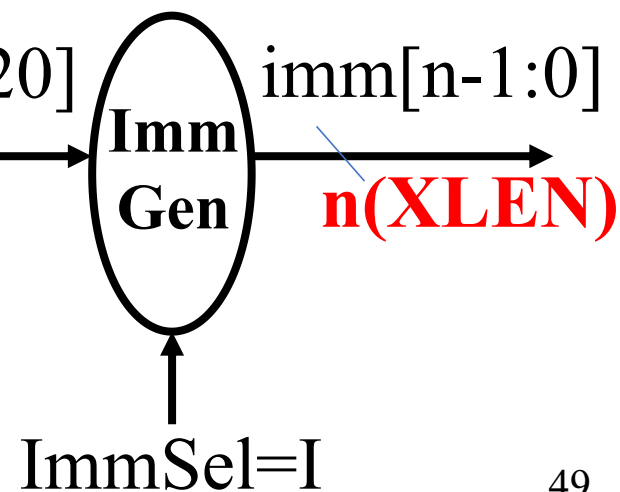
I型指令的立即数生成



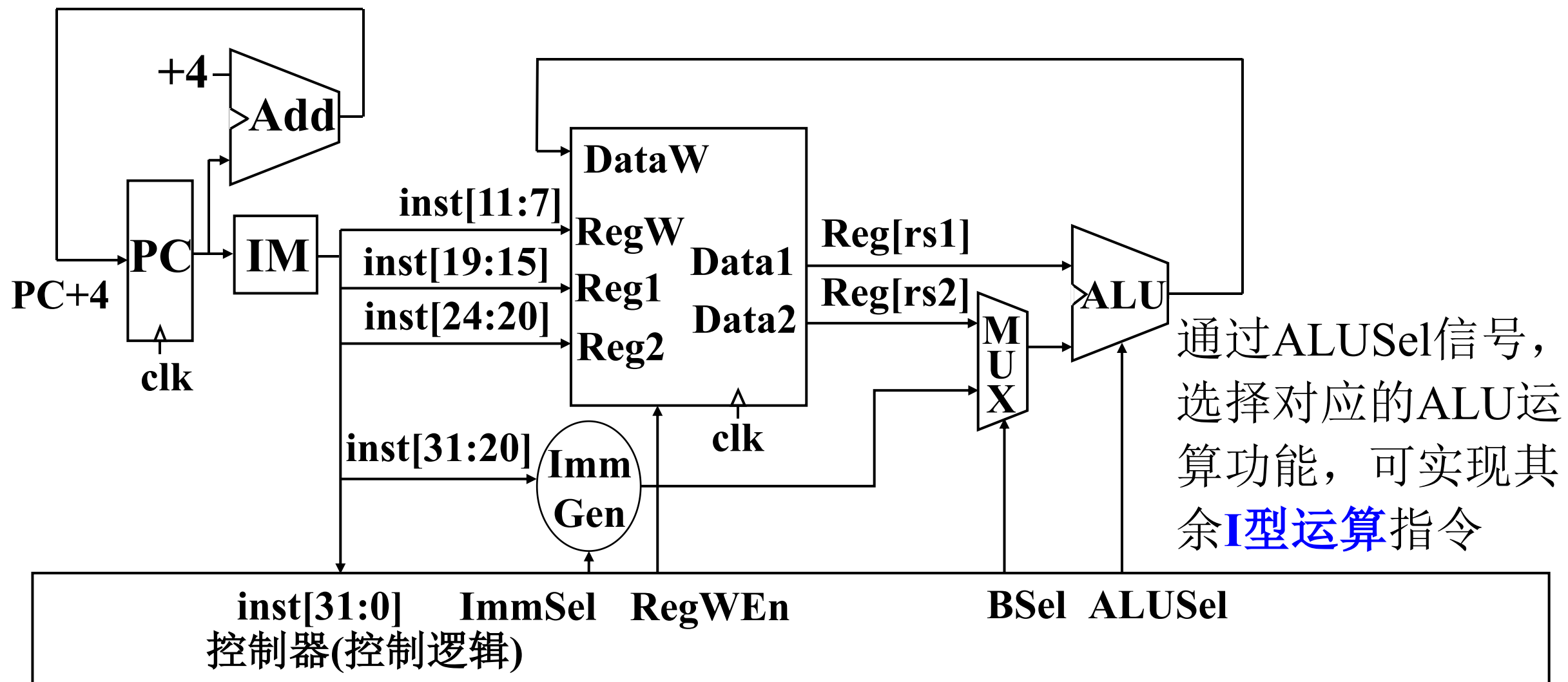
- 指令12位立即数字段复制到立即数的低12位

- 通过将指令的最高比特位复制填充到 $\text{inst}[31:20]$ 到 **Imm Gen** 单元，生成 $\text{imm}[n-1:0]$ ，其中 $n(\text{XLEN})$ 表示立即数的总位数。

立即数除了低12位之外的高位完成符号扩展



R型和addi(I型)共用的数据通路



实现ld(load dword)指令(I型)



ld x14, 8(x2)

00000000	01000	00010	011	01110	0000011
----------	-------	-------	-----	-------	---------

- 寄存器rs1保存基地址，加上立即数，得到目标访问地址
 - 与addi类似，但ALU用于计算地址，而不是获得最终结果
- 从**存储器读**出的数据装入寄存器rd中

RISC-V 访存装载指令

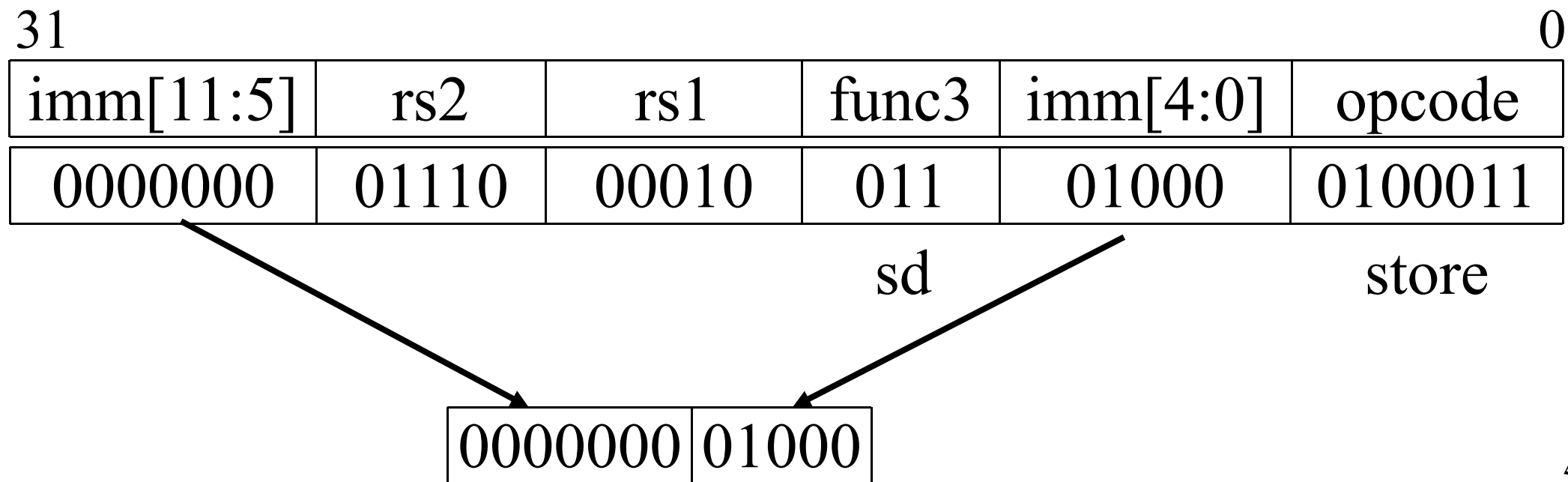
imm[11:0]	rs1	000	rd	0000011	lb
imm[11:0]	rs1	001	rd	0000011	lh
imm[11:0]	rs1	010	rd	0000011	lw
imm[11:0]	rs1	011	rd	0000011	ld
imm[11:0]	rs1	100	rd	0000011	lbu
imm[11:0]	rs1	101	rd	0000011	lhu
imm[11:0]	rs1	110	rd	0000011	lwu

- 为了支持8位、16位、32位等的访存装载指令
 - 增加额外的逻辑电路，用于从存储器取出的数据中提取出字节、半字、字等不同大小的数据
 - 写回到寄存器前，进行符号扩展或零扩展

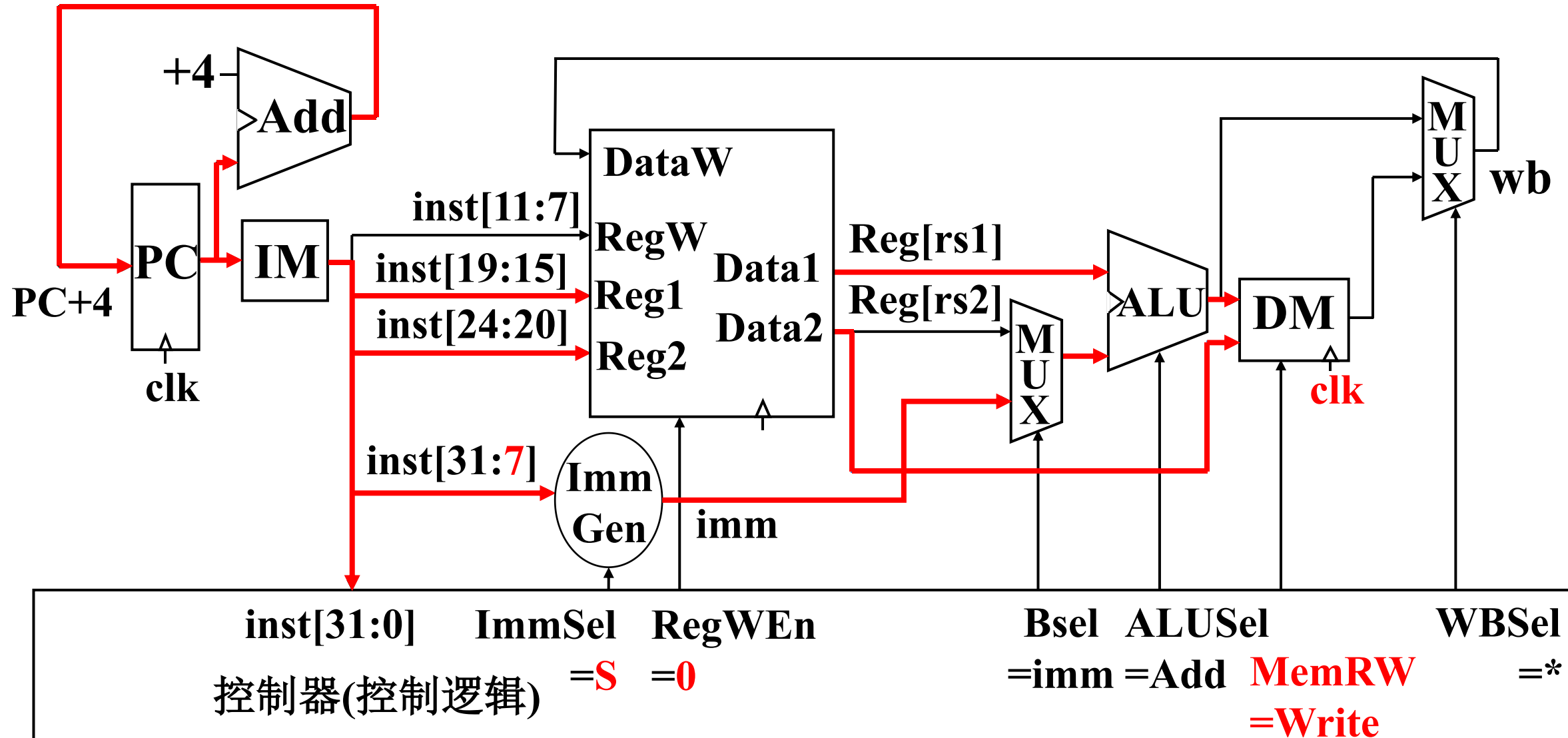
实现S型指令——sd

- 读取两个寄存器，rs1作为提供基地址的源寄存器，rs2作为提供待保存数据的源寄存器，以及立即数偏移量

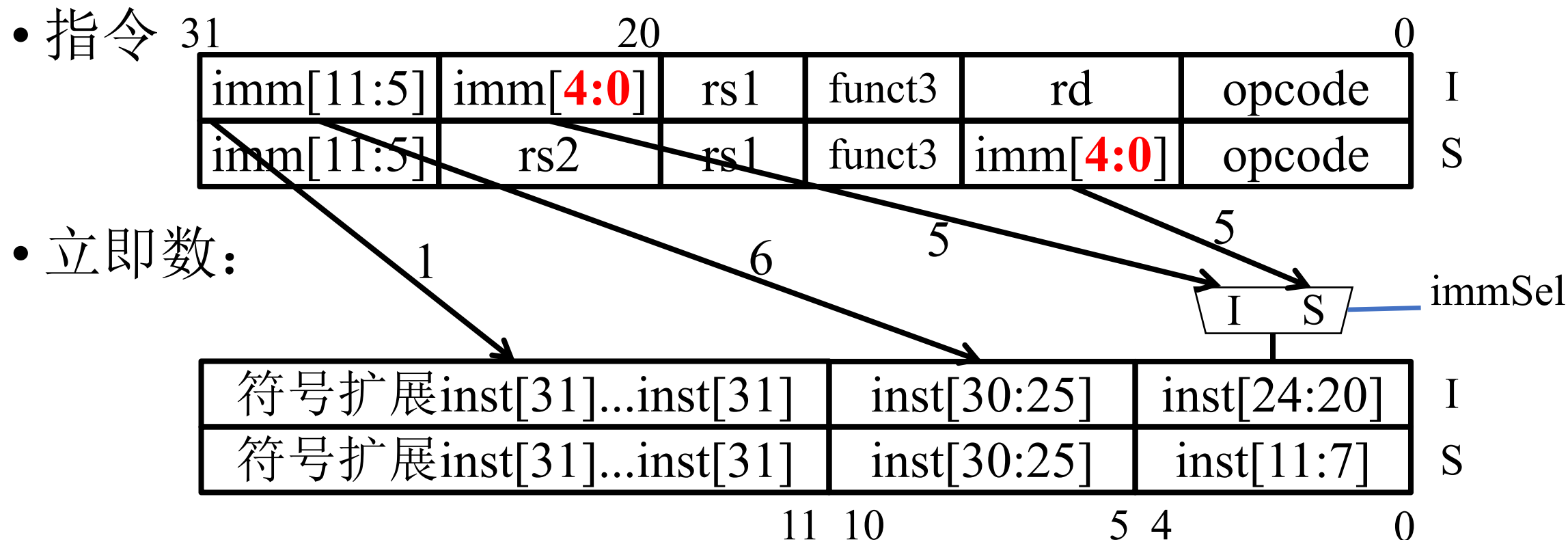
sd x14, 8(x2)



sd指令的数据通路

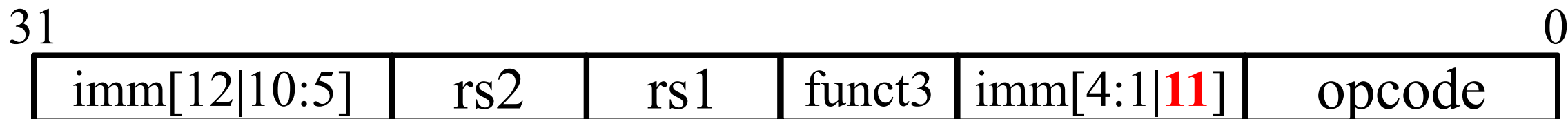


I型指令和S型指令中的立即数生成



- 立即数低5位由I型或S型对应的控制信号选择
- 立即数中的其他位连接到指令中的固定位置

实现B型指令

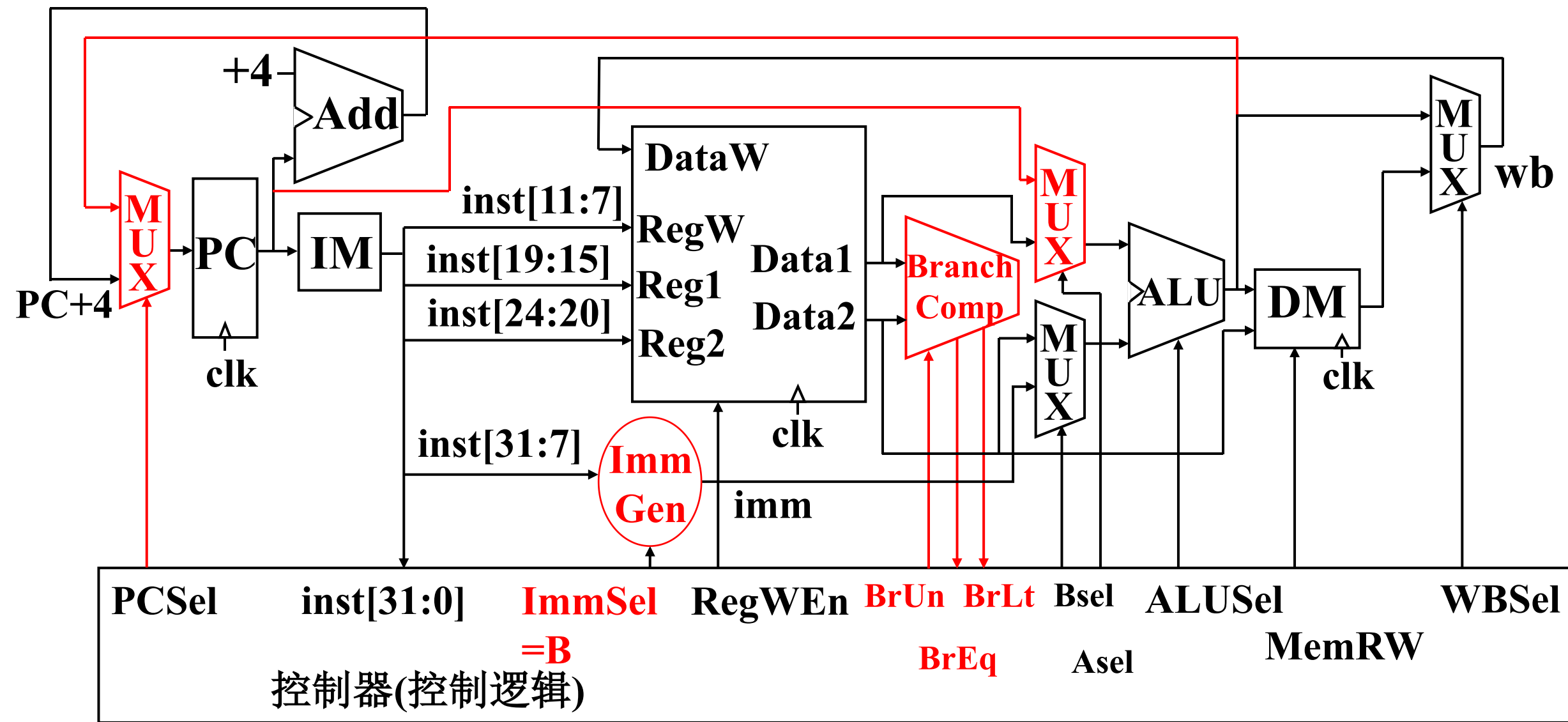


- B型指令格式与S型指令格式基本相同
- 但立即数字段以2字节为增量表示-4096到+4094的偏移量
- 12位立即数字段表示13位有符号字节地址的偏移量
 - 偏移量的最低位始终为零，因此无需存储

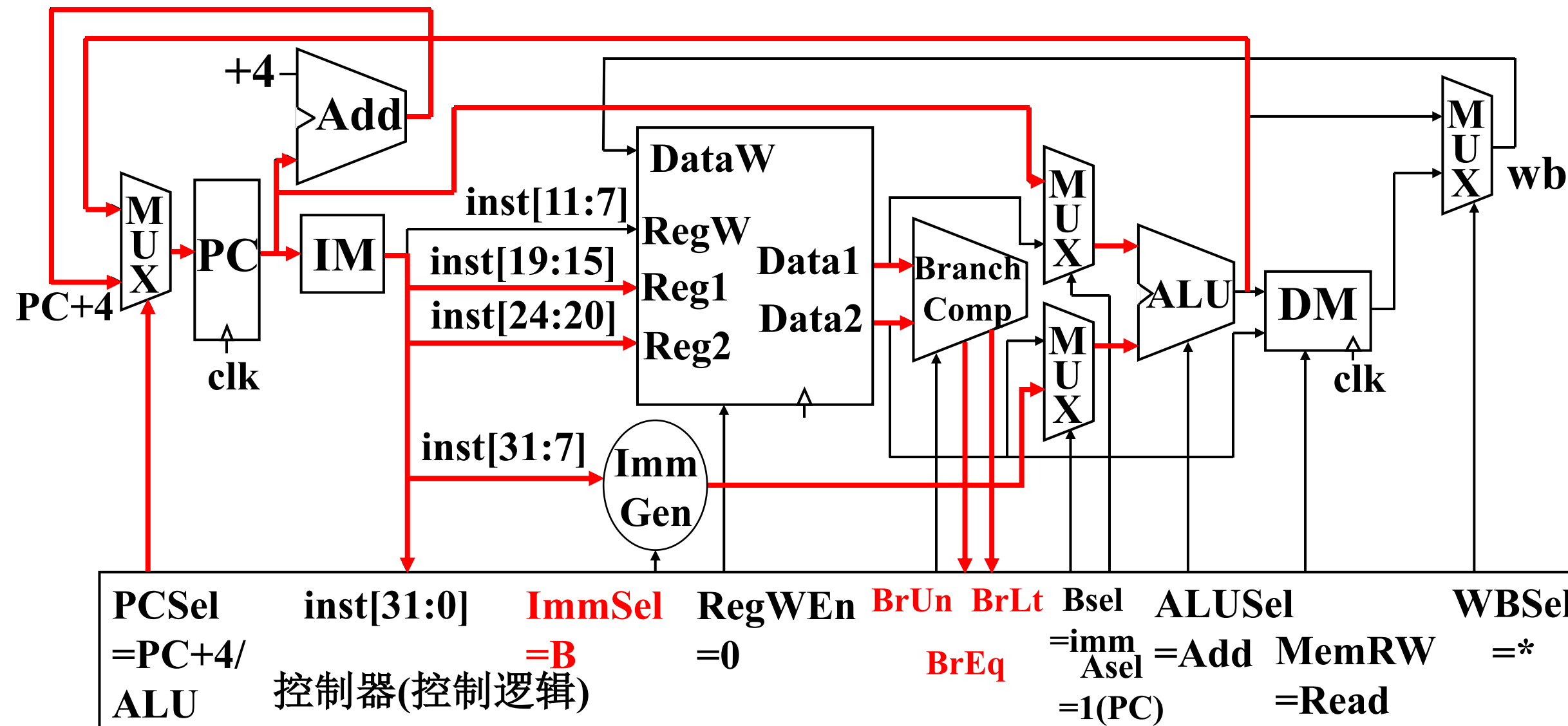
B型指令的数据通路

- B型指令： beq、 bne、 blt、 bge、 bltu、 bgeu
- PC不同的状态变化：
 - $PC + 4$ (不发生分支转移)
 - $PC + \text{immediate}$ (发生分支转移)
- 需要比较rs1和rs2的数值关系，并计算 $PC + \text{立即数}$ 的结果
- 加一个ALU vs. 加专门的比较硬件电路

B型指令的数据通路

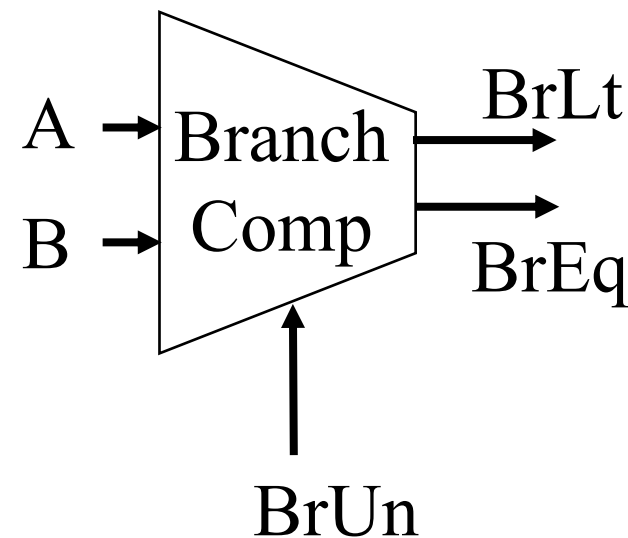


B型指令的数据通路



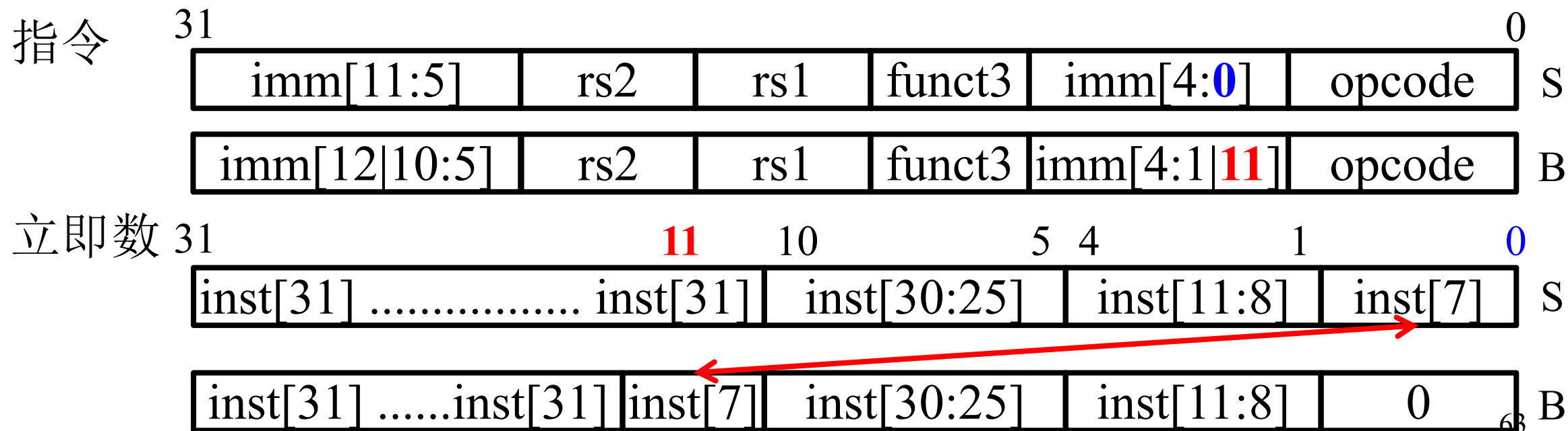
分支跳转比较器

- 当 $A = B$ 时，输出 $BrEq = 1$ ，否则为0
- 当 $A < B$ 时，输出 $BrLt = 1$ ，否则为0
- 输入 $BrUn = 1$ 时，进行无符号比较
输入 $BrUn = 0$ 时，进行有符号比较
- 对于bge，可以根据 $BrLt$ 信号取反判断 $A \geq B$

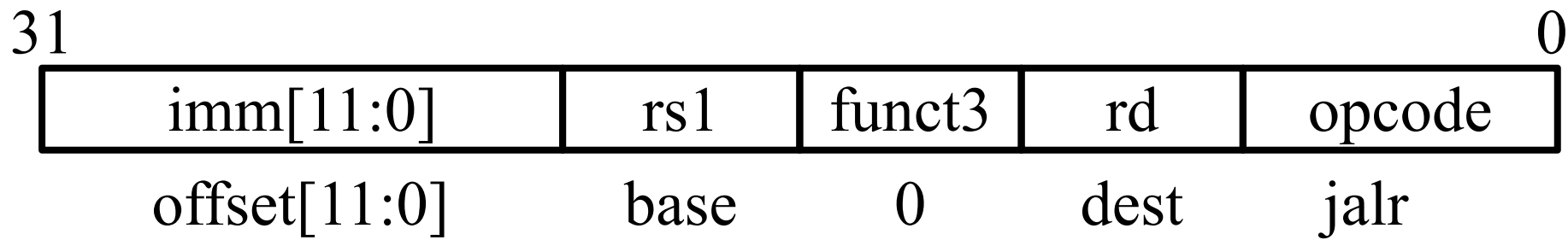


B型指令立即数生成

- 与S型指令立即数生成方式类似
- S型立即数最低位在B型立即数中变为第12位
- 只有一位数据在编码位置上有差异
 - 需要两个1位两路选择器

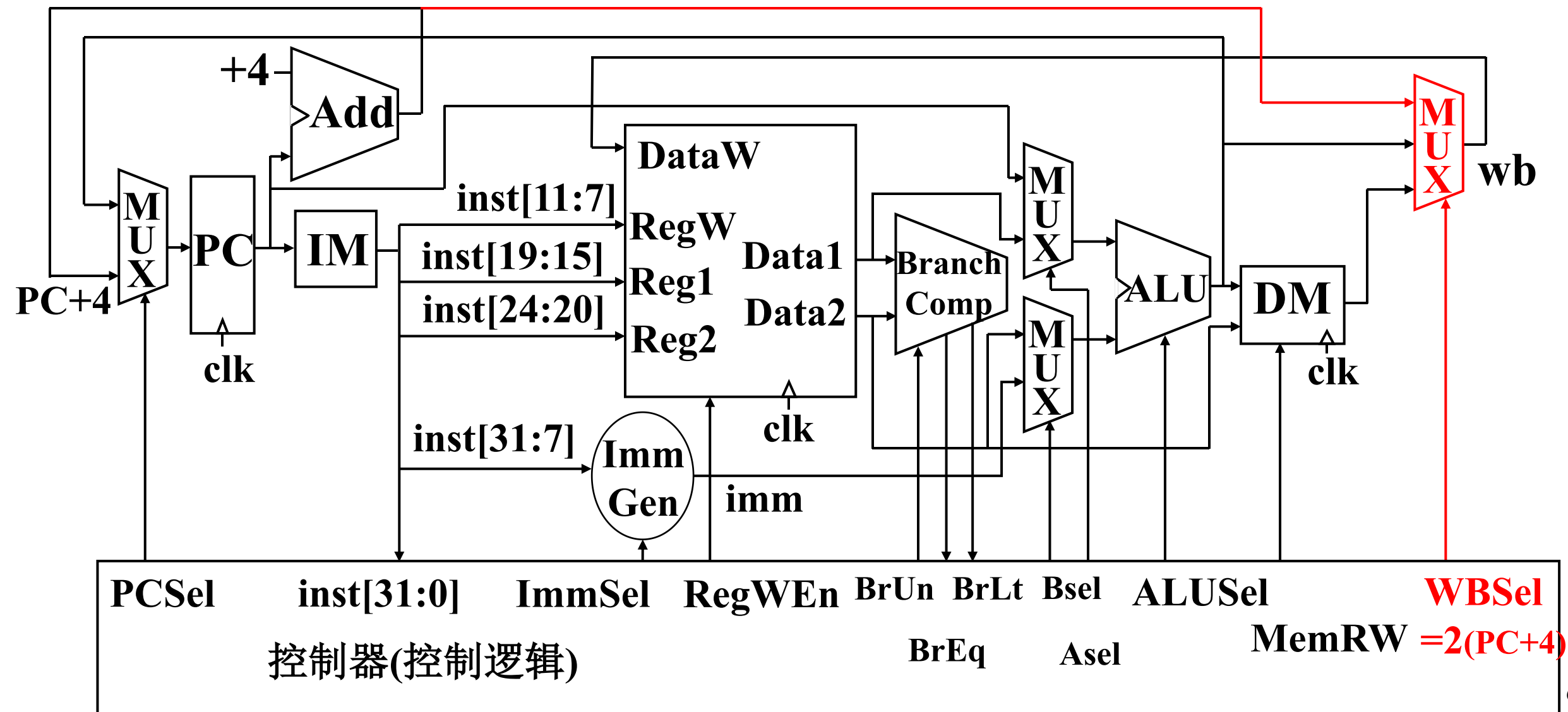


I型指令——jalr

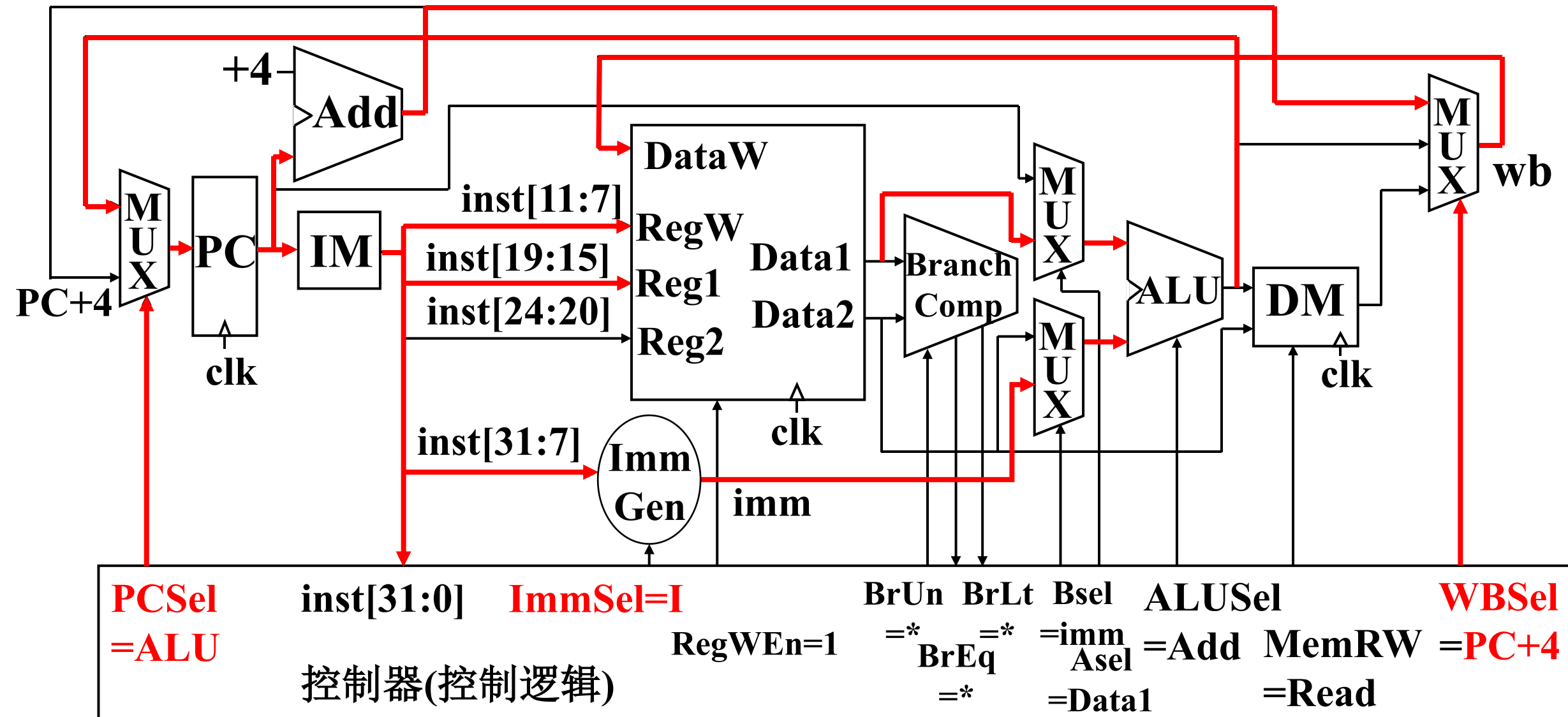


- jalr rd, offset(rs1) #(jump and link register)
- 将 $PC + 4$ 写入 **rd** (返回地址)
- 设置 **$PC = R[rs1] + offset$**
- 立即数生成与I型算术和装载指令一样

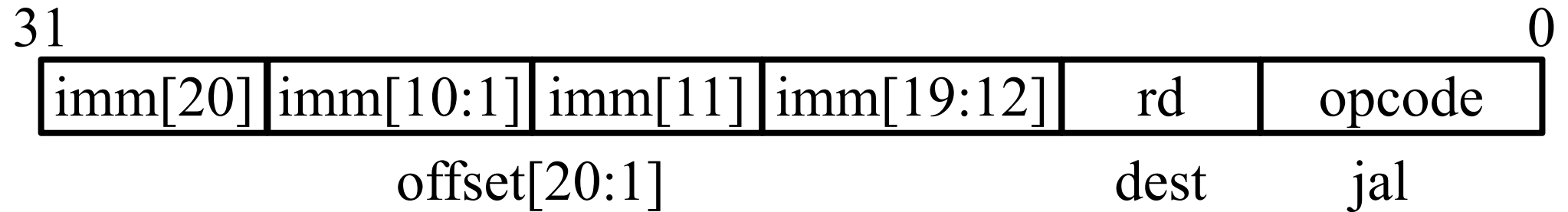
jalr指令的数据通路



jalr指令的数据通路

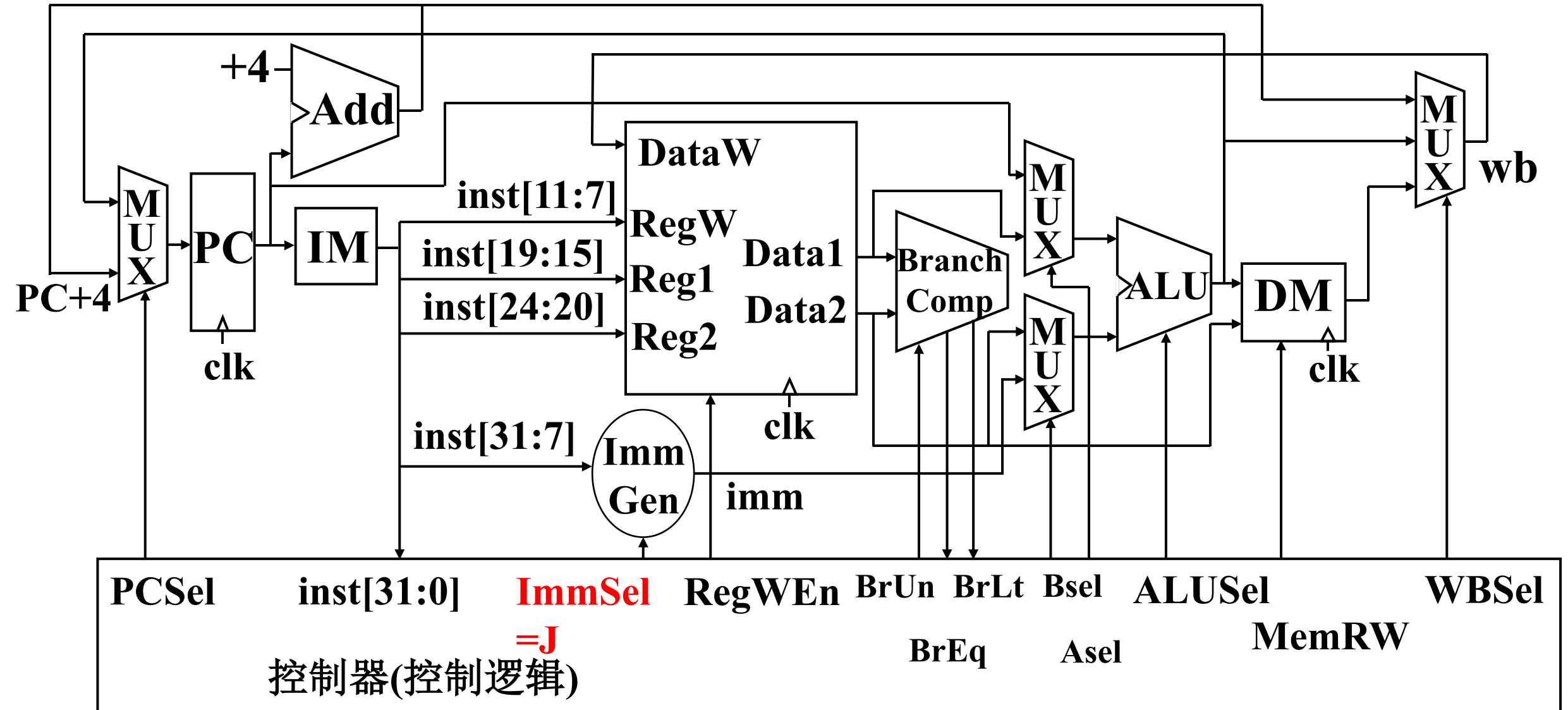


J型指令——jal

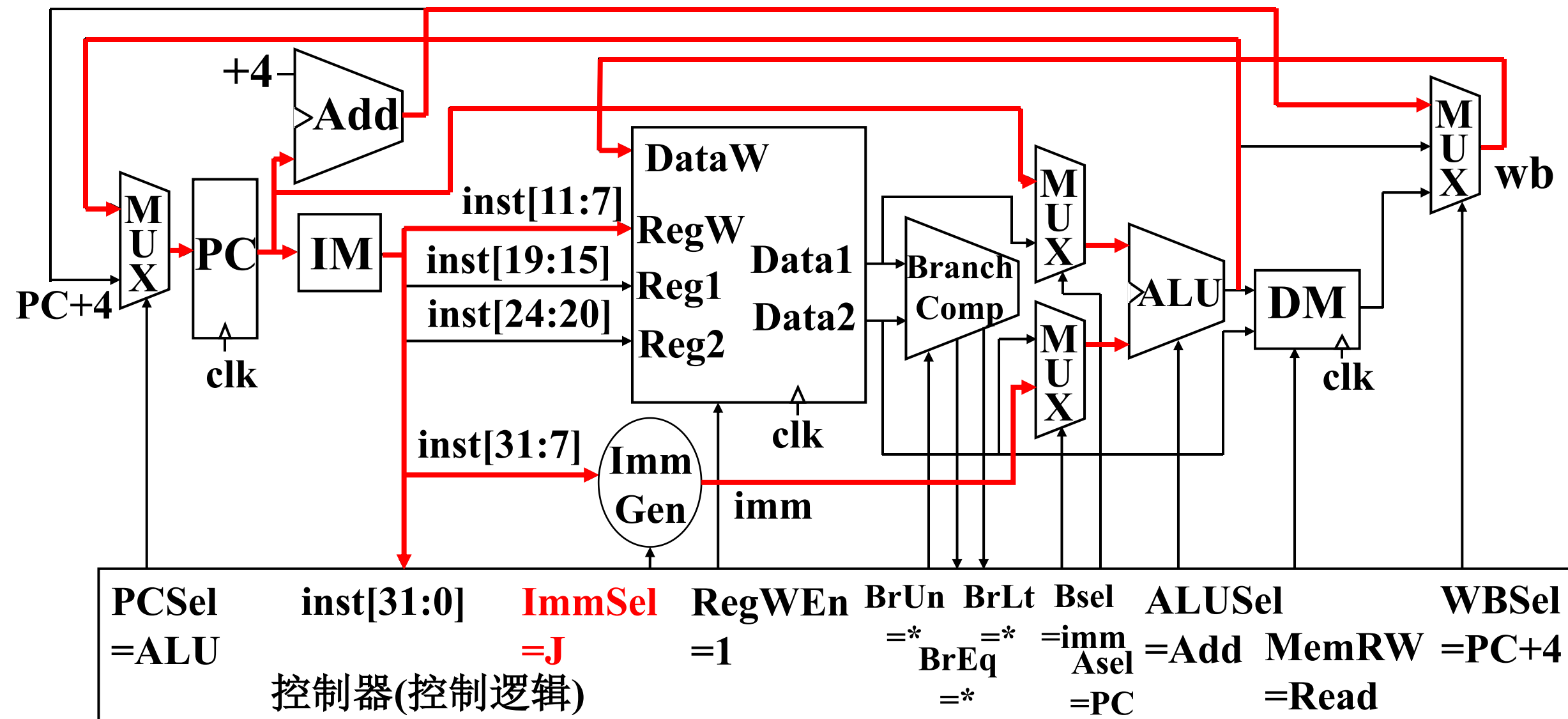


- jal将 $PC + 4$ 写入目的寄存器rd中
- 设置 $PC = PC + offset$
- 立即数字段20位，对应一个 $\pm 2^{19}$ 的偏移量(以**2字节**为单位)

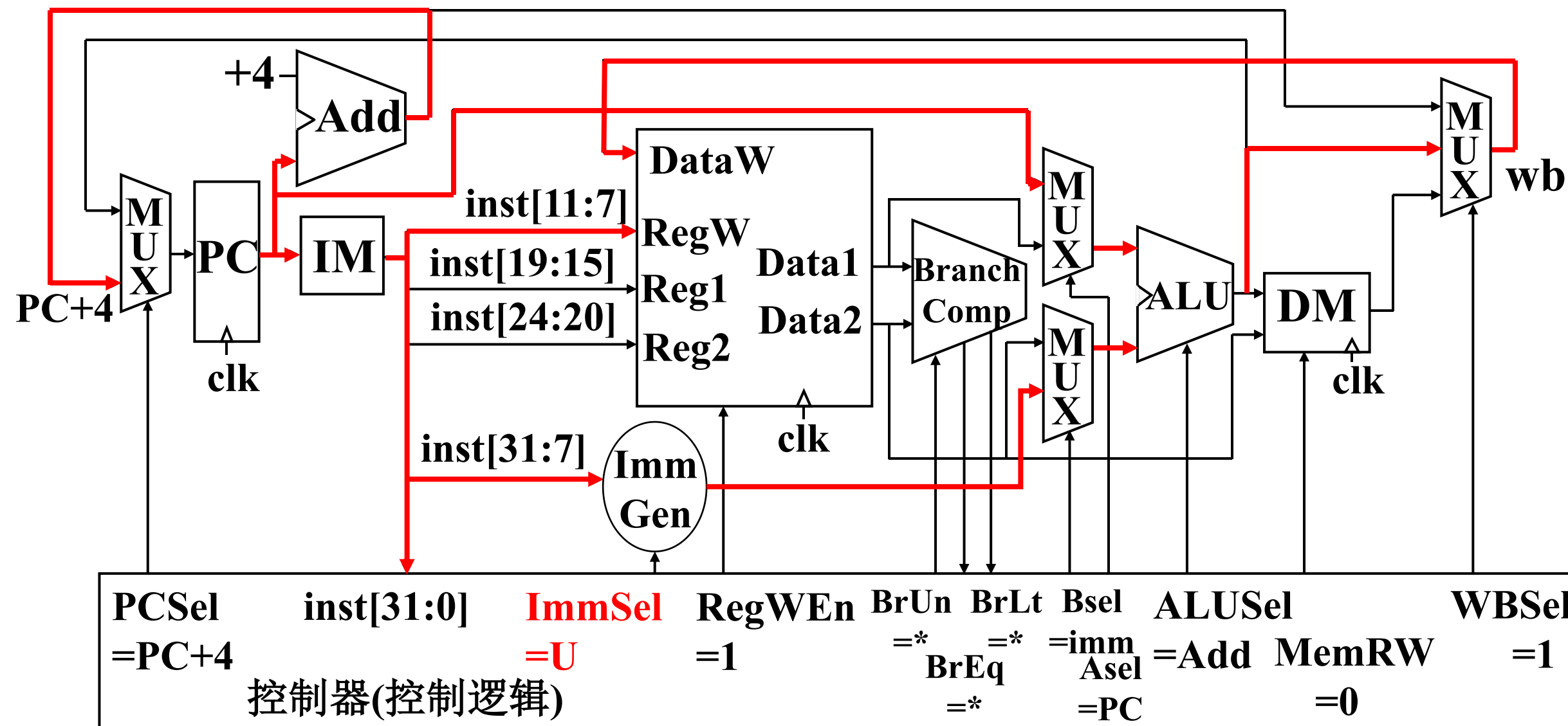
jal指令的数据通路



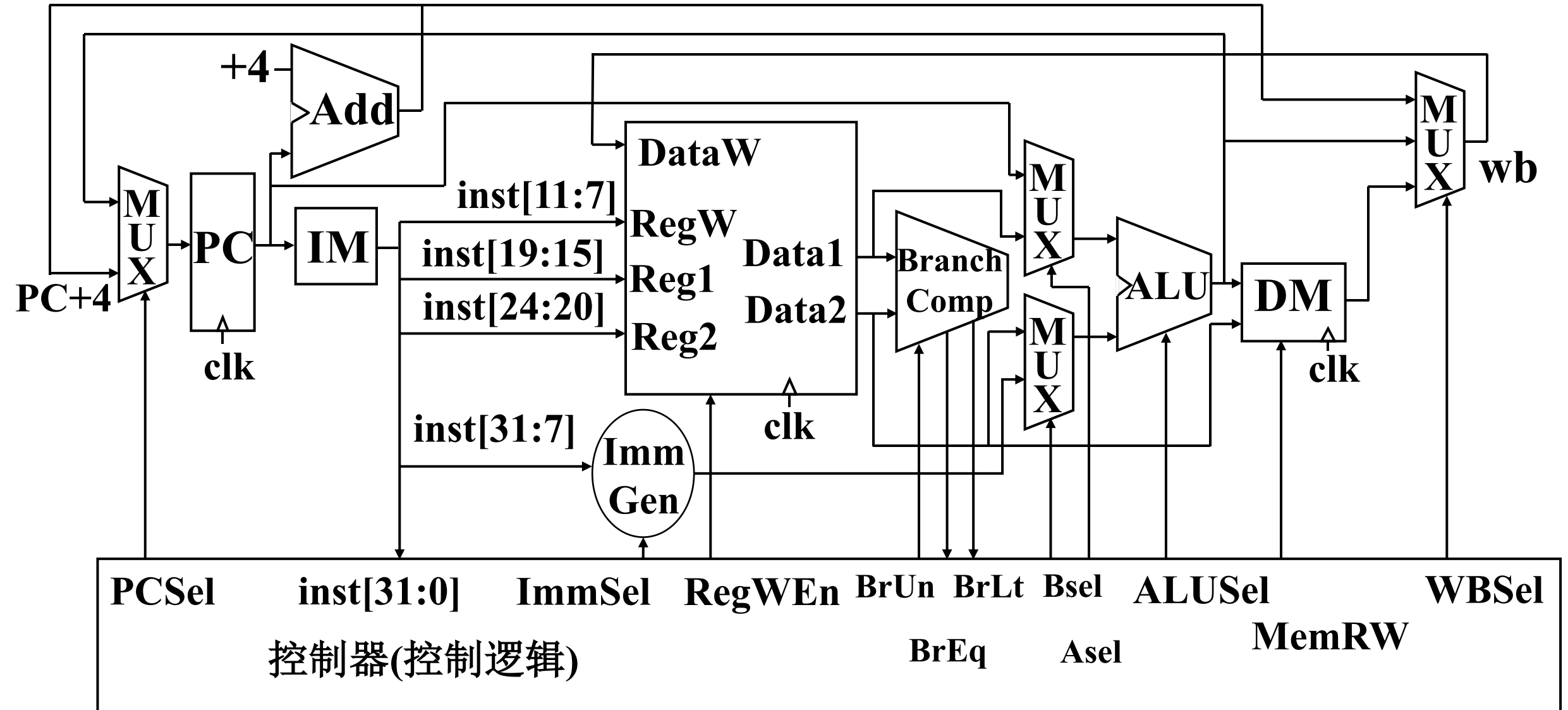
jal指令的数据通路



U型指令auipc的数据通路

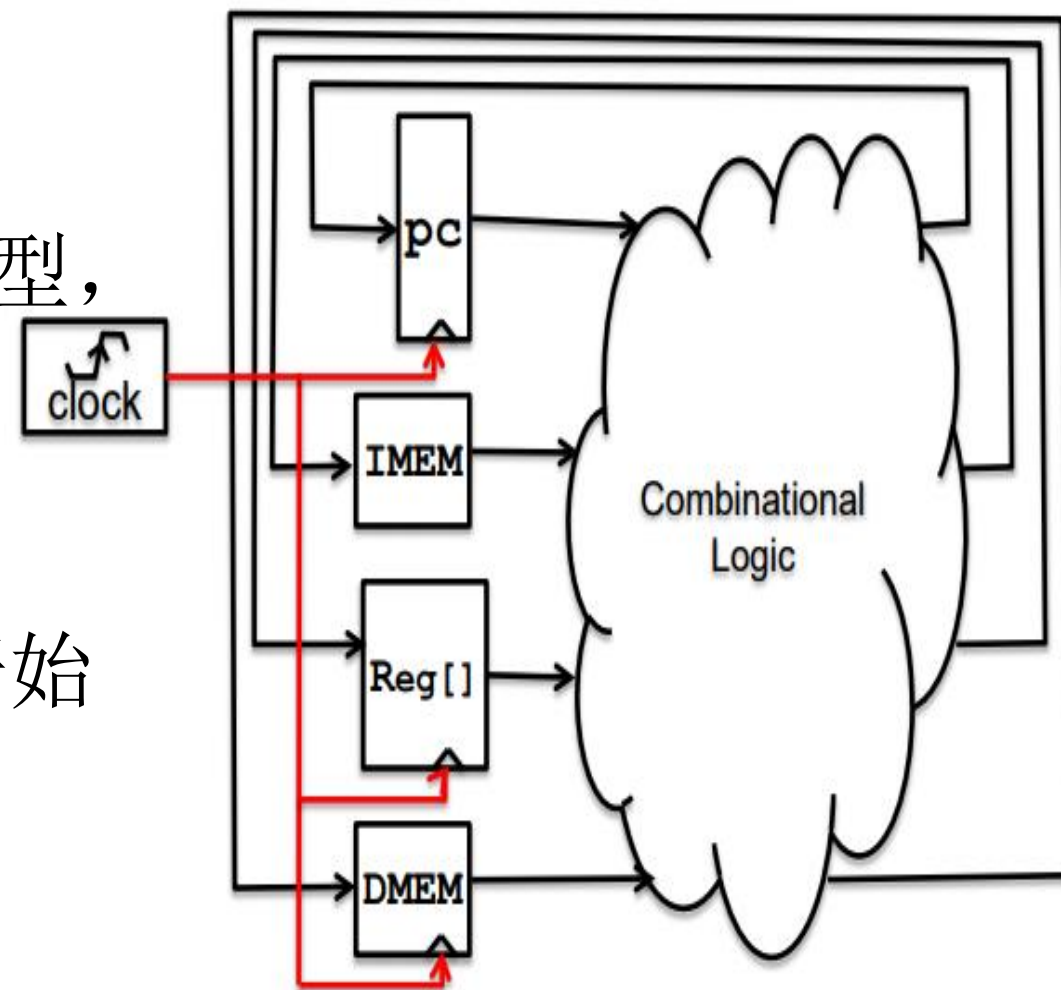


实现例子中指令的RISC-V 数据通路



单周期CPU设计模型

- 每个时钟周期执行一条指令
- 当前状态值通过一系列组合逻辑得到指令机器码，以及操作码类型，操作数，运算结果等。
- 在下一有效时钟沿到来时，所有时序逻辑单元状态更新，同时开始下一个时钟周期的操作。



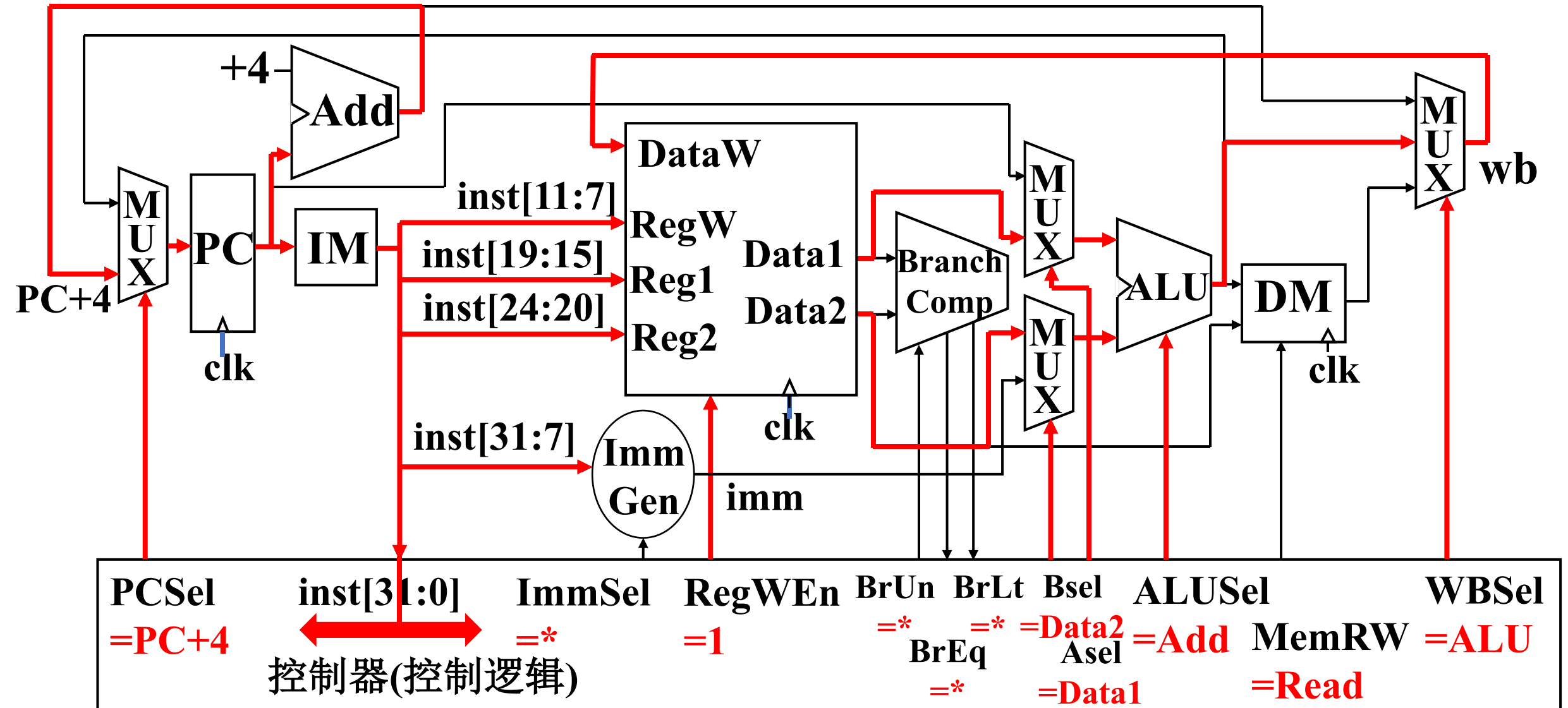
第4章 处理器设计

- 处理器设计的需求分析
- RISC-V数据通路的组件选择
- RISC-V部分指令的数据通路设计
- **RISC-V控制器**

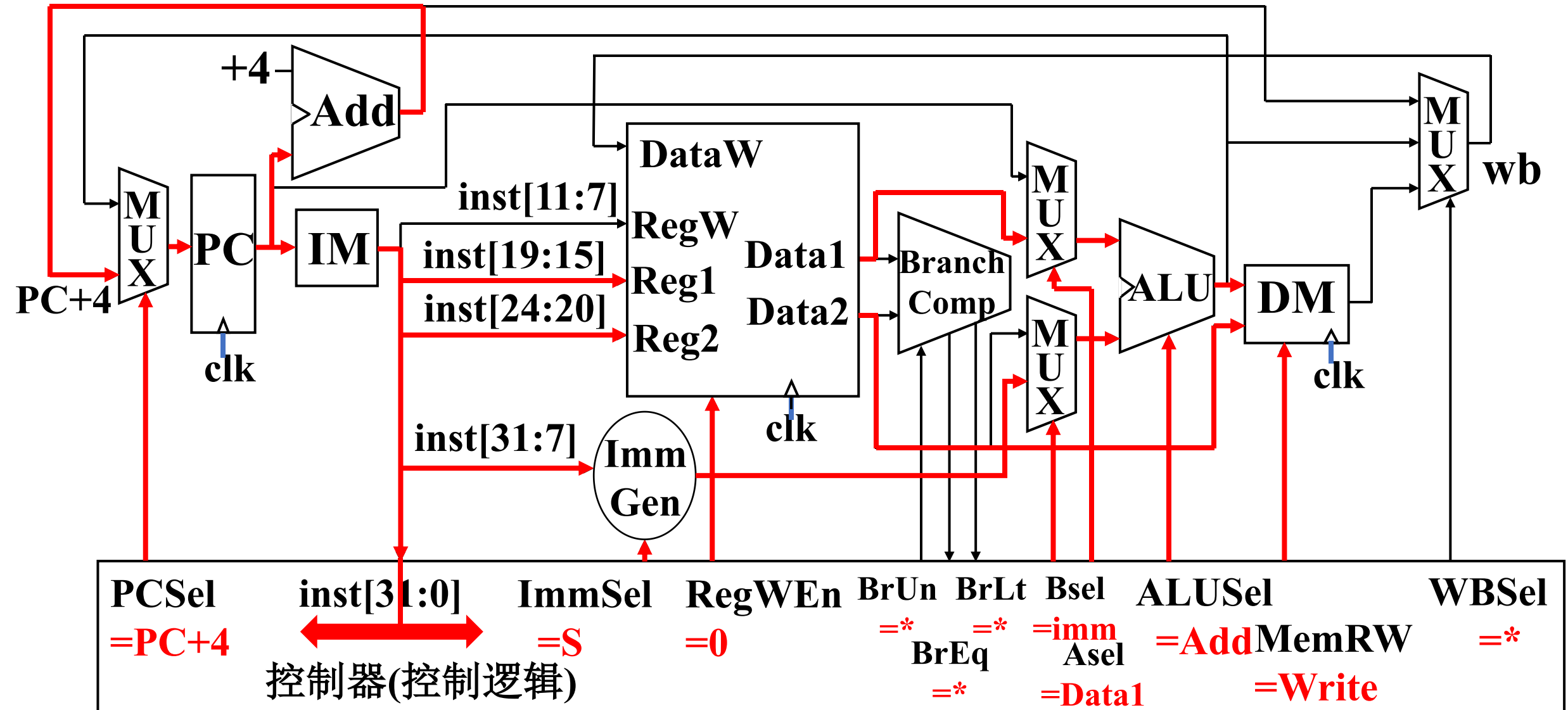
处理器的设计步骤

- ① 分析指令系统，得出对数据通路的需求
- ② 为数据通路选择合适的组件
- ③ 根据指令需求连接组件建立数据通路
- ④ 分析每条指令的实现，以确定控制信号
- ⑤ 集成控制信号，形成完整的控制逻辑

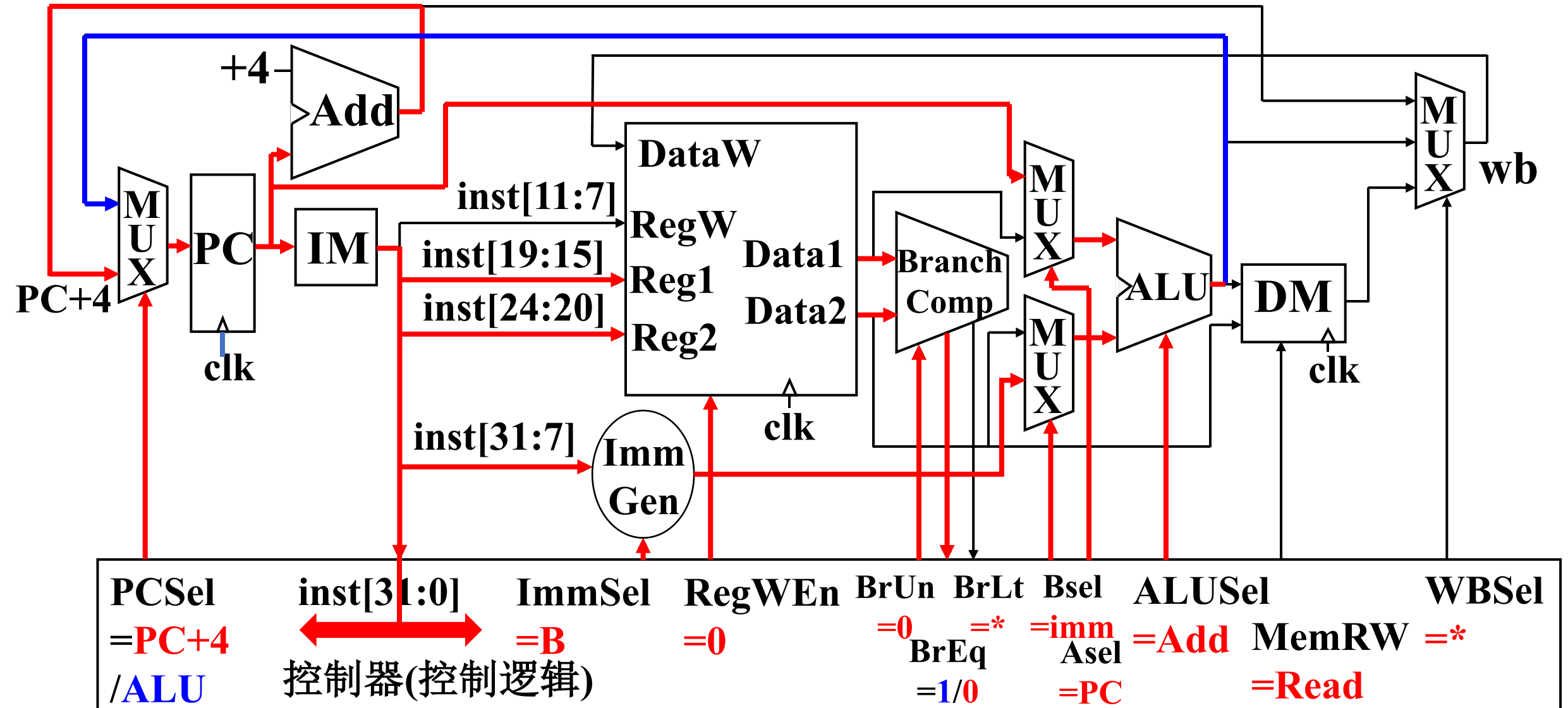
add指令的执行



sd指令的执行



beq指令的执行



RISC-V 编码在控制器中的作用

- 从前述的RISC-V 指令集编码可以看出，实际上用来区分某条指令种类的编码只有**inst[30]**、**inst[14:12]**、**inst[6:2]**

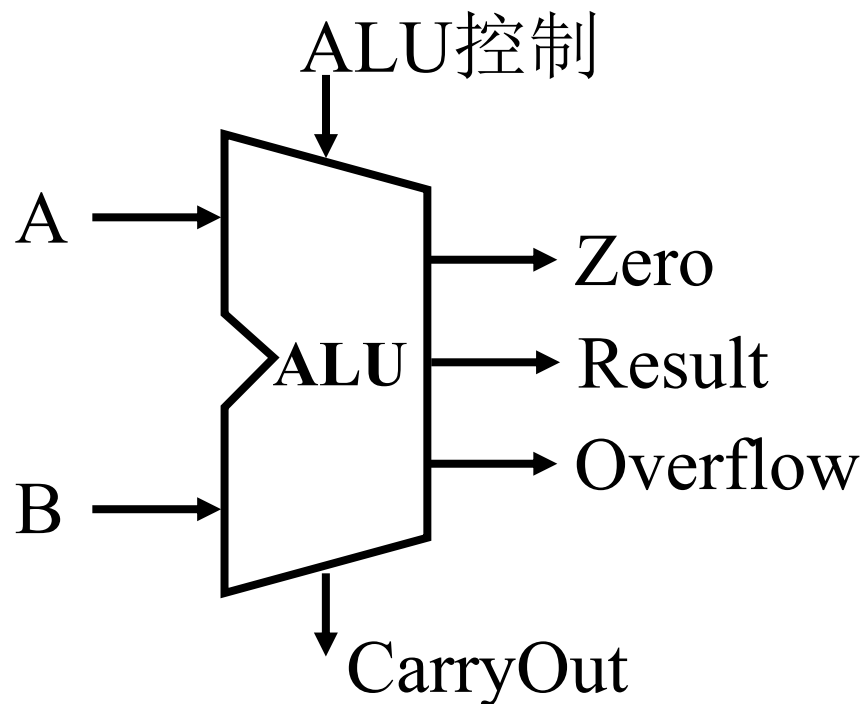
	inst[31:25]	inst[24:20]	inst[19:15]	inst[14:12]	inst[11:7]	inst[6:0]
R型	func7	rs2	rs1	func3	rd	opcode
I型	imm[11 :5]	imm[4:0]	rs1	func3	rd	opcode
S型	imm[11 :5]	rs2	rs1	func3	imm[4:0]	opcode
B型	imm[12 ,10:5]	rs2	rs1	func3	imm[4:1, 11]	opcode
J型	imm[20 ,10:5]	imm[4:1, 11]	imm[19:15]	imm[14:12]	rd	opcode
U型	imm[31 :25]	imm[24:20]	imm[19:15]	imm[14:12]	rd	opcode

组合逻辑控制——例子

Inst[14:12]				Inst[6:2]		
imm[12 10:5]	rs2	rs1		imm[4:1 11]		
			↓		↓	
imm[12 10:5]	rs2	rs1	000	imm[4:1 11]	1100011	BEQ
imm[12 10:5]	rs2	rs1	001	imm[4:1 11]	1100011	BNE
imm[12 10:5]	rs2	rs1	100	imm[4:1 11]	1100011	BLT
imm[12 10:5]	rs2	rs1	101	imm[4:1 11]	1100011	BGE
imm[12 10:5]	rs2	rs1	110	imm[4:1 11]	1100011	BLTU
imm[12 10:5]	rs2	rs1	111	imm[4:1 11]	1100011	BGEU

$$\text{BrUn} = \text{Inst}[14] \cdot \text{Inst}[13] \cdot \text{Inst}[6] \cdot \text{Inst}[5] \cdot \text{Inst}[4]' \cdot \text{Inst}[3]' \cdot \text{Inst}[2]'$$

ALU功能需求及设计思路



ALU控制	操作
0000	与(and)
0001	或(or)
0010	加(add)
0110	减(sub)
0111	小于则置位(slt)
1100	或非(nor)

- 从简单开始：然后再进行扩展
- 分而治之：从1位ALU设计开始
- 利用已知的元件或者组件，组合起来实现复杂功能

根据opcode和ALUOp控制位设置ALU控制

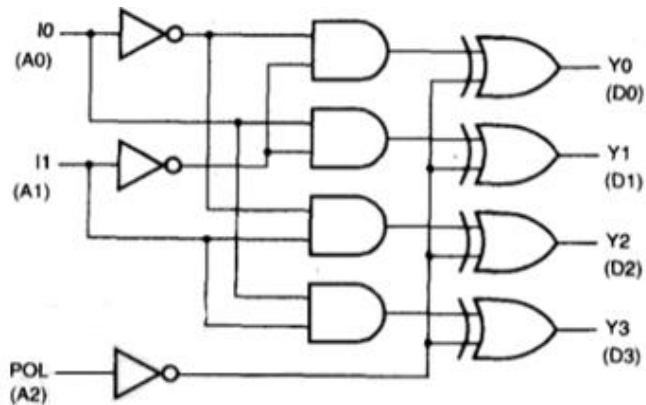
opcode	ALUOp	操作	func7	func3	期望ALU操作	ALU控制输入
ld	00	load dword	*****	***	add	0010
sd	00	store dword	*****	***	add	0010
beq	01	branch if equal	*****	***	add	0010
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

ALU控制输入信号真值表

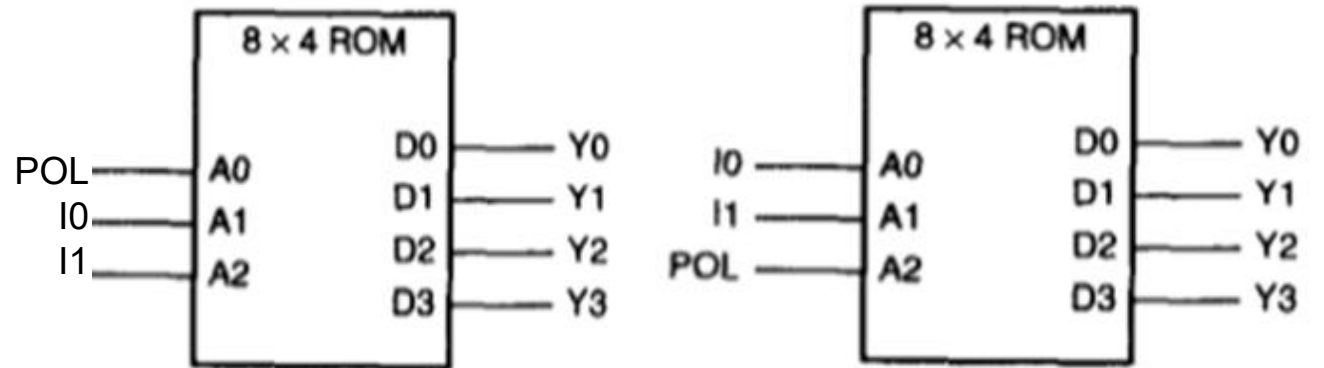
ALUOp		funct7字段							funct3字段			ALU控制输入
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	
0	0	x	x	x	x	x	x	x	x	x	x	0010
0	1	x	x	x	x	x	x	x	x	x	x	0010
1	0	0	0	0	0	0	0	0	0	0	0	0010
1	0	0	1	0	0	0	0	0	0	0	0	0110
1	0	0	0	0	0	0	0	0	1	1	1	0000
1	0	0	0	0	0	0	0	0	1	1	0	0001

数字逻辑电路的不同实现方式

- 使用分立的逻辑门
- 使用ROM
 - 使用ROM的物理实现并不是唯一的。
- 使用FPGA



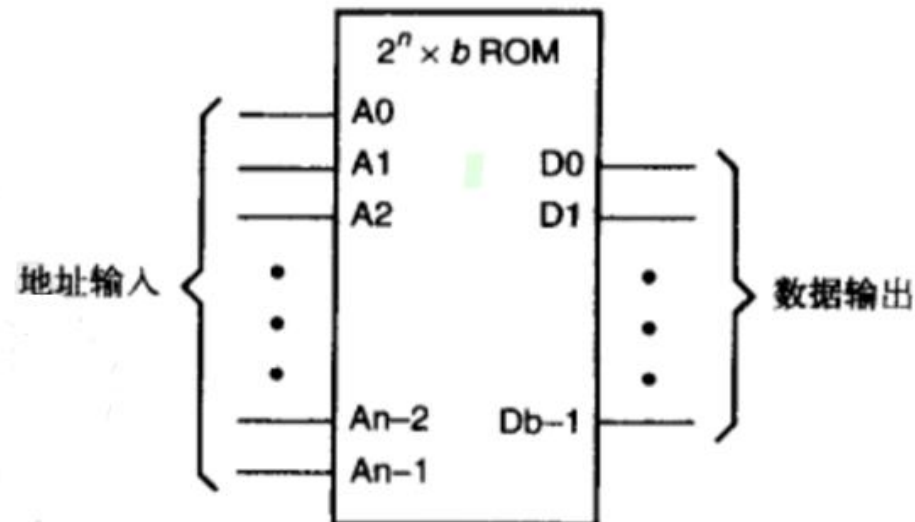
具有输出极性控制的2-4译码器



用ROM构建具有极性控制的2-4译码器

ROM (Read-Only Memory)

- ROM是一种具有 n 个输入 b 个输出的组合逻辑电路。
- 输入被称为地址输入 (address input)，通常命名为 $A_0, A_1, \dots, A_{n-1}$ 。
- 输出被称为数据输出 (data output)，通常命名为 $D_0, D_1, \dots, D_{b-1}$ 。



一个 $2^n \times b$ ROM的基本结构

ROM和真值表

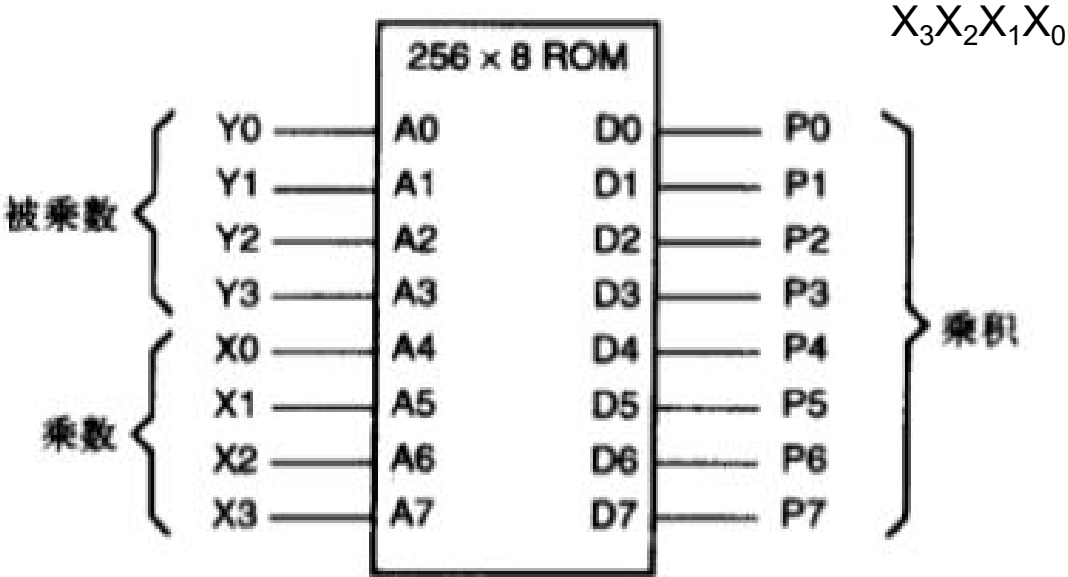
- ROM “存储” 了一个n输入、b输出的组合逻辑功能的真值表。
- 一个3输入、4输出的组合功能的真值表，可以被存储在一个 $2^3 * 4$ ($8 * 4$) 的只读存储器中
- 忽略延迟，ROM的数据输出总是等于真值表中由地址输入所选择的那行输出位。

输入			输出			
A2	A1	A0	D3	D2	D1	D0
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

一个3输入4输出组合逻辑函数的真值表
(具有输出极性控制的2-4译码器)

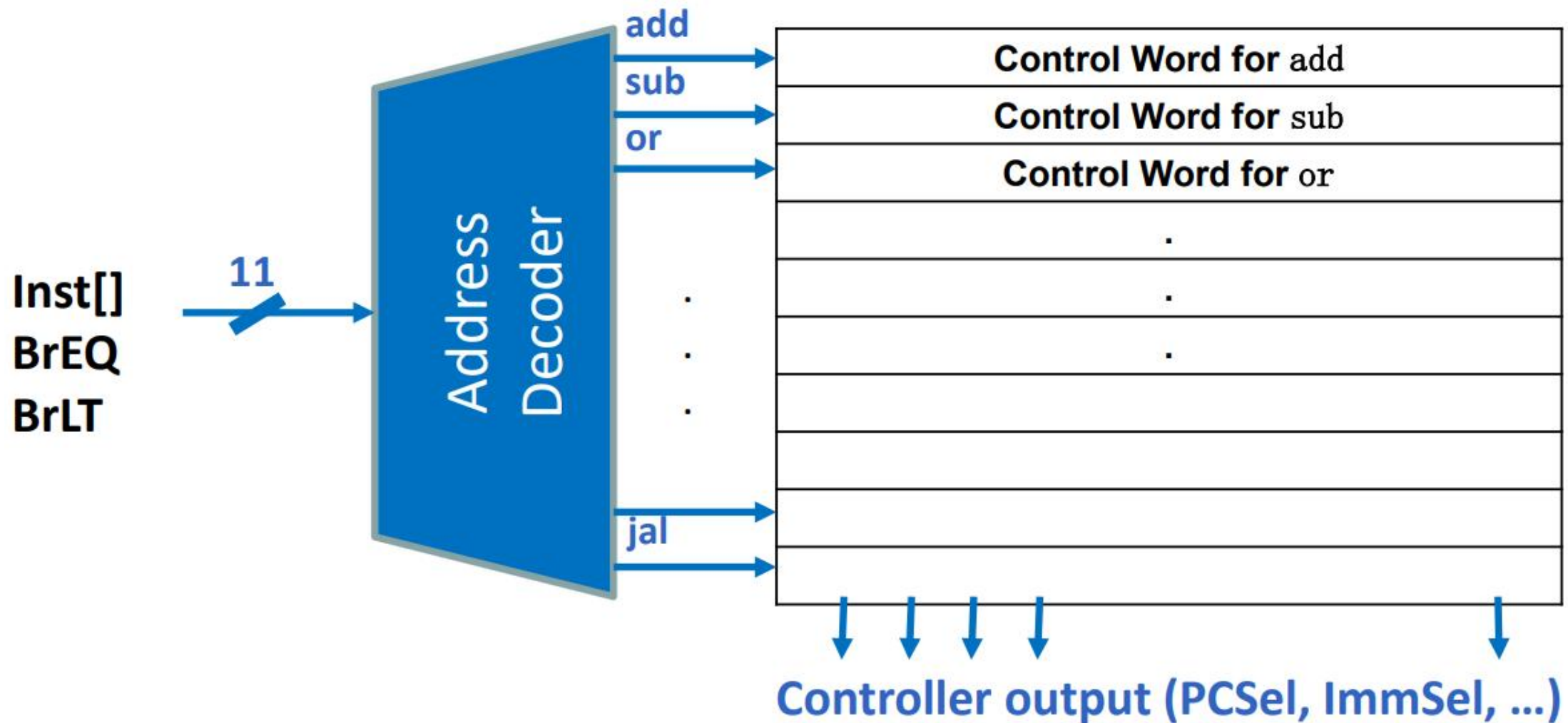
用ROM实现4位*4位无符号二进制数乘法

- 多少种组合？
- 乘积最多为几位？

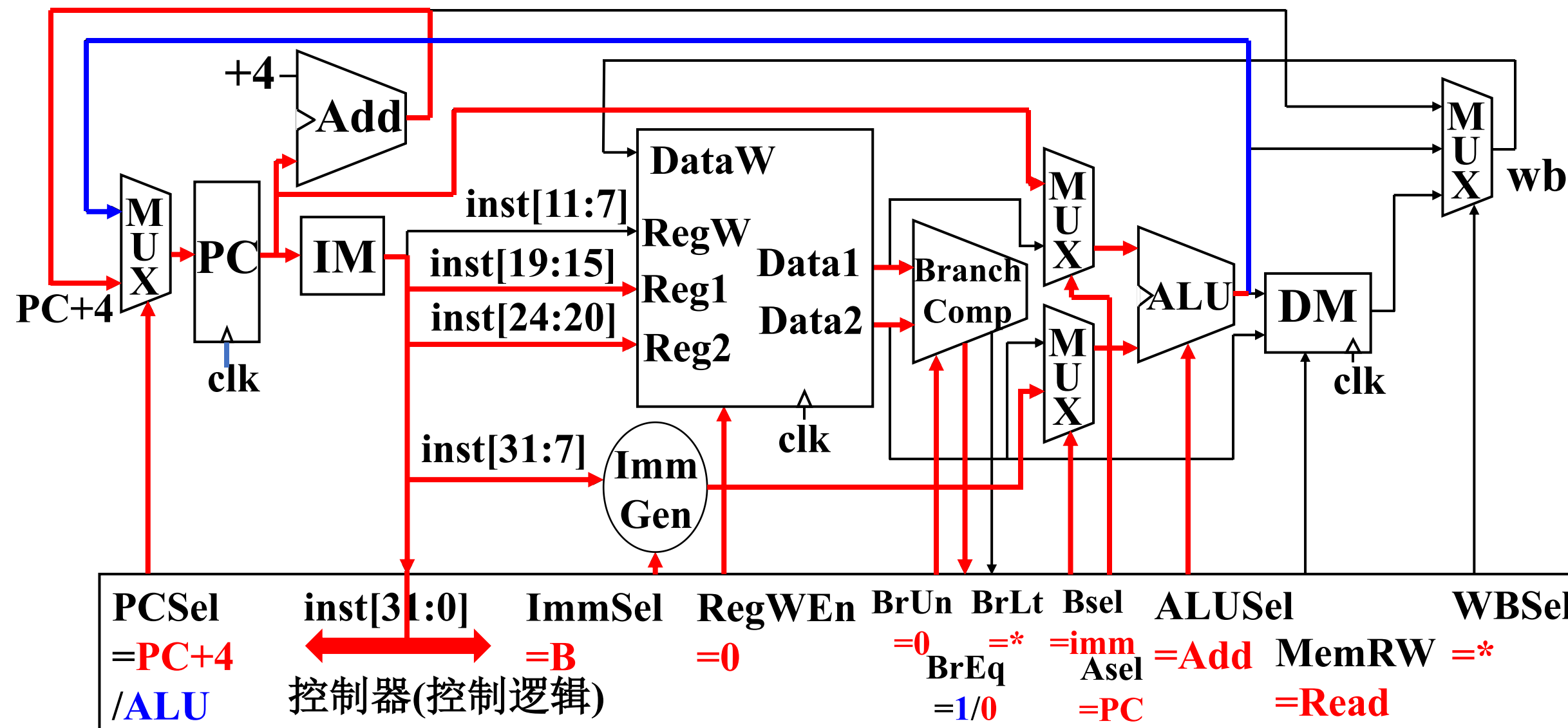


地址	Y ₃ Y ₂ Y ₁ Y ₀															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00
10	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F
20	00	02	04	06	08	0A	0C	0E	10	12	14	16	18	1A	1C	1E
30	00	03	06	09	0C	0F	12	15	18	1B	1E	21	24	27	2A	2D
40	00	04	08	0C	10	14	18	1C	20	24	28	2C	30	34	38	3C
50	00	05	0A	0F	14	19	1E	23	28	2D	32	37	3C	41	46	4B
60	00	06	0C	12	18	1E	24	2A	30	36	3C	42	48	4E	54	5A
70	00	07	0E	15	1C	23	2A	31	38	3F	46	4D	54	5B	62	69
80	00	08	10	18	20	28	30	38	40	48	50	58	60	68	70	78
90	00	09	12	1B	24	2D	36	3F	48	51	5A	63	6C	75	7E	87
A0	00	0A	14	1E	28	32	3C	46	50	5A	64	6E	78	82	8C	96
B0	00	0B	16	21	2C	37	42	4D	58	63	6E	79	84	8F	9A	A5
C0	00	0C	18	24	30	3C	48	54	60	6C	78	84	90	9C	A8	B4
D0	00	0D	1A	27	34	41	4E	5B	68	75	82	8F	9C	A9	B6	C3
E0	00	0E	1C	2A	38	46	54	62	70	7E	8C	9A	A8	B6	C4	D2
F0	00	0F	1E	2D	3C	4B	5A	69	78	87	96	A5	B4	C3	D2	E1

基于ROM的控制器实现



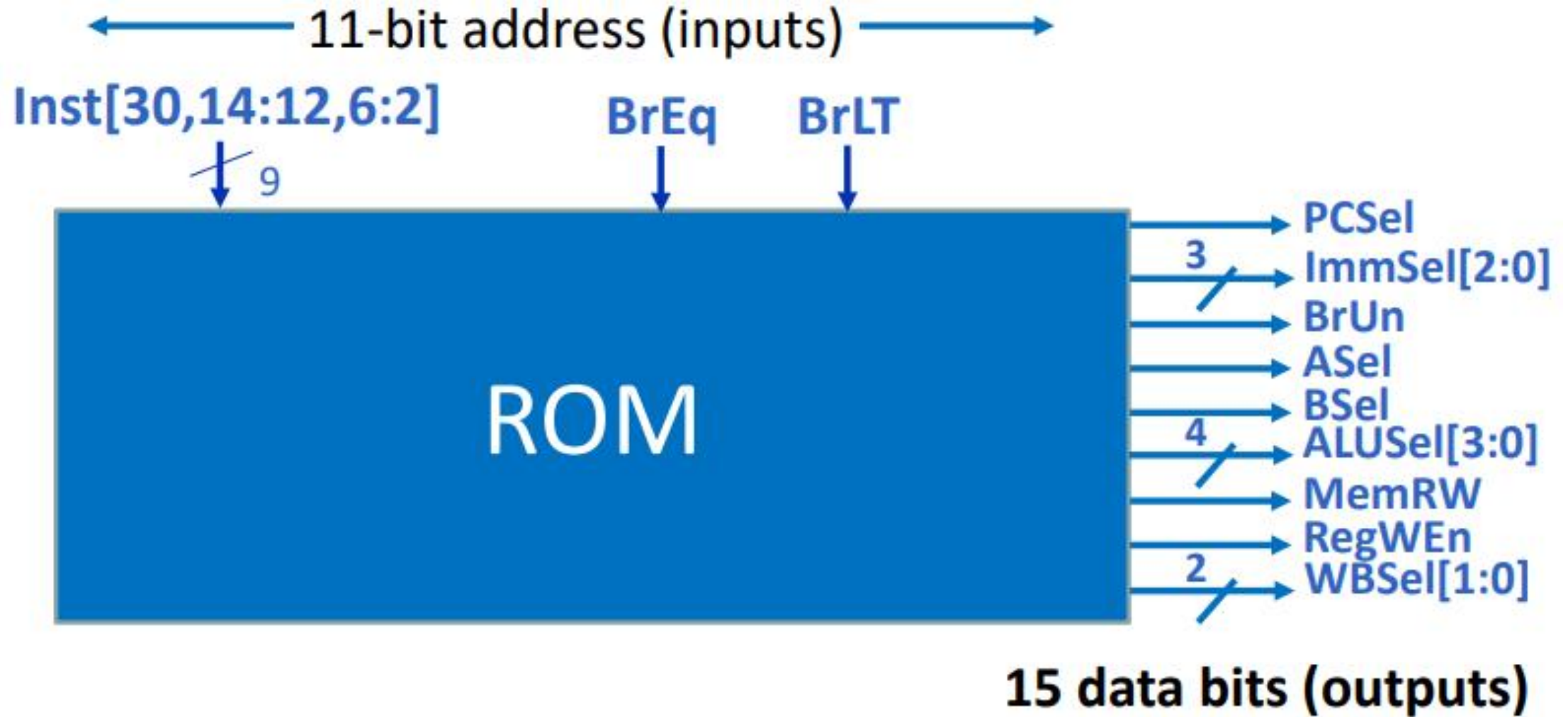
RISC V指令的执行



控制信号真值表

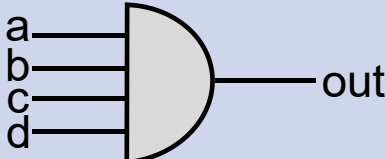
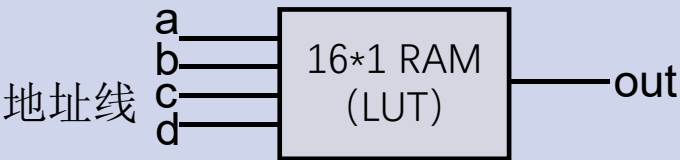
Inst[31:0]	BrEq	BrLt	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWen	WBSel
add	*	*	+4	*	*	Reg	Reg	Add	Read	1	ALU
sub	*	*	+4	*	*	Reg	Reg	Add	Read	1	ALU
addi	*	*	+4	I	*	Reg	Imm	Add	Read	1	ALU
ld	*	*	+4	I	*	Reg	Imm	Add	Read	1	Mem
sd	*	*	+4	S	*	Reg	Imm	Add	Write	0	*
beq	0	*	+4	B	*	PC	Imm	Add	Read	0	*
beq	1	*	ALU	B	*	PC	Imm	Add	Read	0	*
...	...	...	...	...	...	...	...	...			
blt	*	1	ALU	B	0	PC	Imm	Add	Read	0	*
bltu	*	1	ALU	B	1	PC	Imm	Add	Read	0	*
jalr	*	*	ALU	I	*	Reg	Imm	Add	Read	1	PC+4
jal	*	*	ALU	J	*	PC	Imm	Add	Read	1	PC+4
auipc	*	*	+4	U	*	PC	Imm	Add	Read	1	ALU

ROM 控制



FPGA中的查找表（LUT）

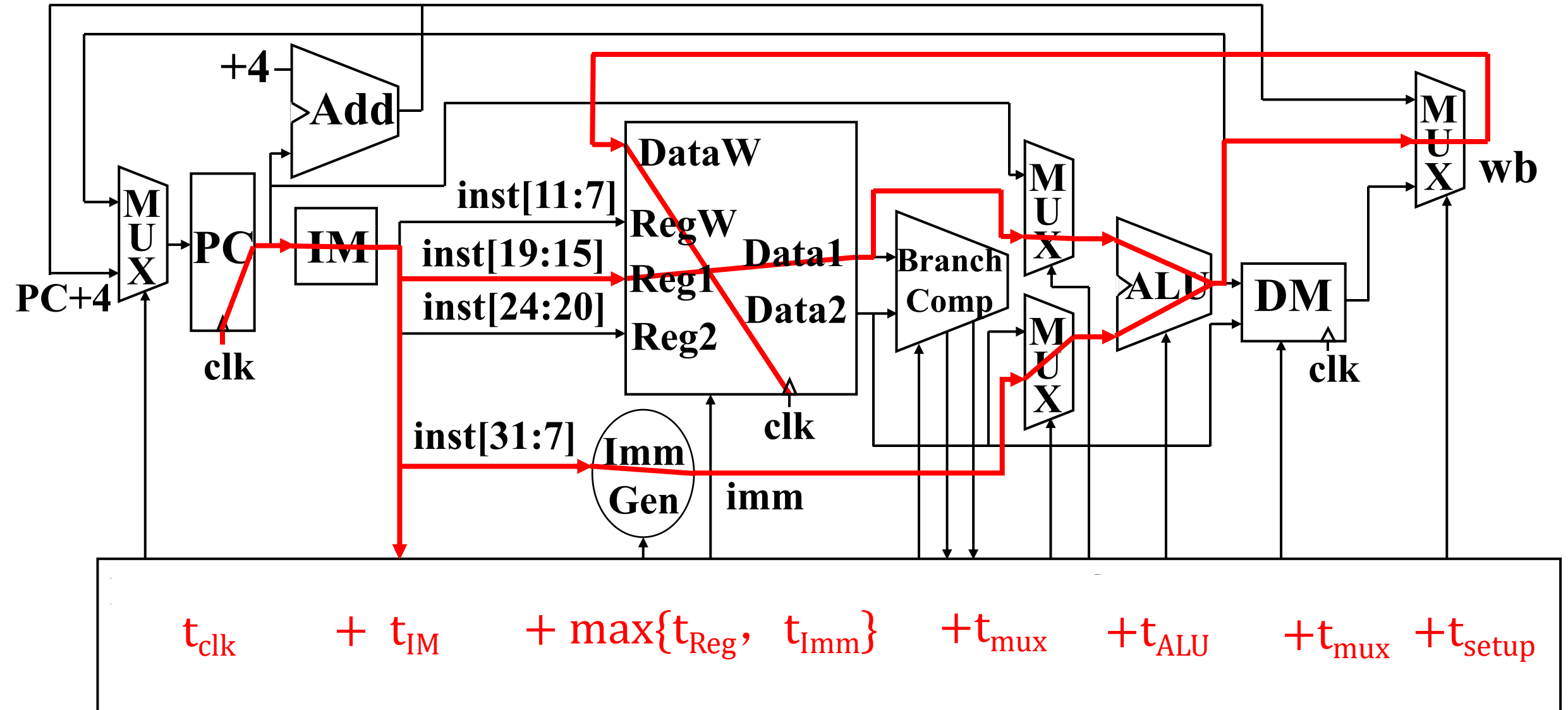
- 例：使用LUT实现一个4与门电路逻辑功能

实际逻辑电路		LUT的实现方式	
			
a、b、c、d	逻辑输出	地址	RAM中存储的内容
0000	0	0000	0
0001	0	0001	0
.....	0		0
1111	1	1111	1

LUT本质就是RAM，主流的FPGA中LUT一般是5输入或6输入。

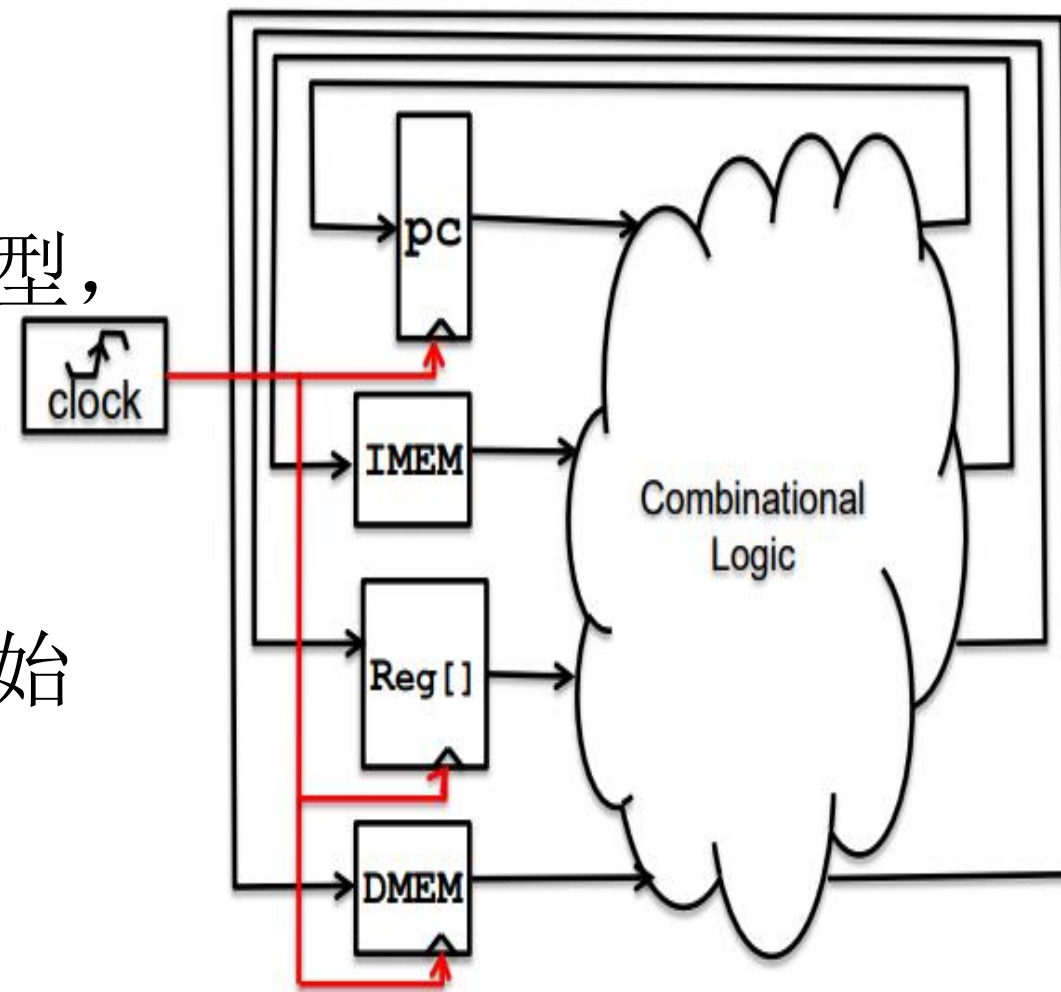
a,b,c,d由FPGA芯片的管脚输入后进入可编程连线，然后作为地址线连到LUT，LUT中已经事先写入了所有可能的逻辑结果，通过地址查找到相应的数据然后输出，这样组合逻辑就实现了。

关键路径（以R型指令为例）

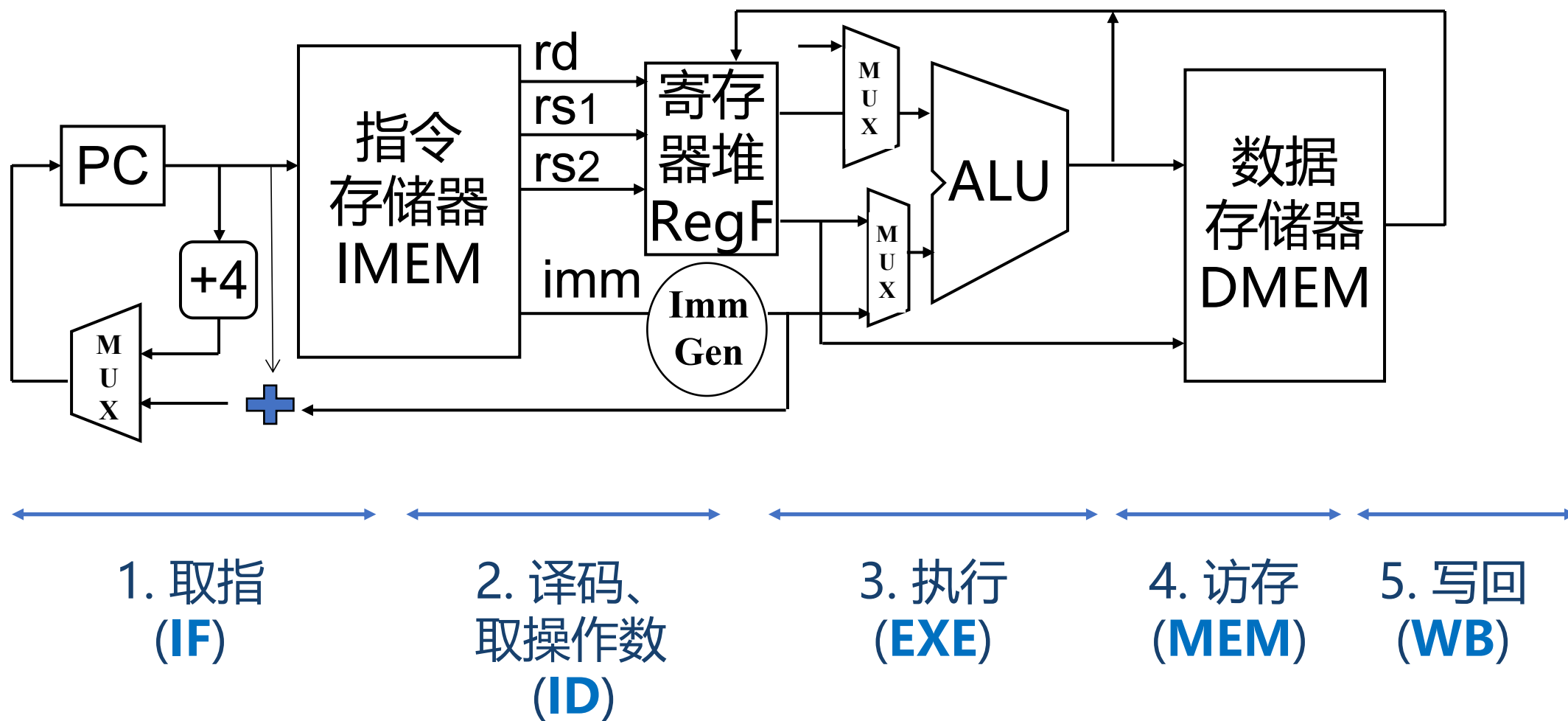


单周期CPU设计模型

- 每个时钟周期执行一条指令
- 当前状态值通过一系列组合逻辑得到指令机器码，以及操作码类型，操作数，运算结果等。
- 在下一有效时钟沿到来时，所有时序逻辑单元状态更新，同时开始下一个时钟周期的操作。



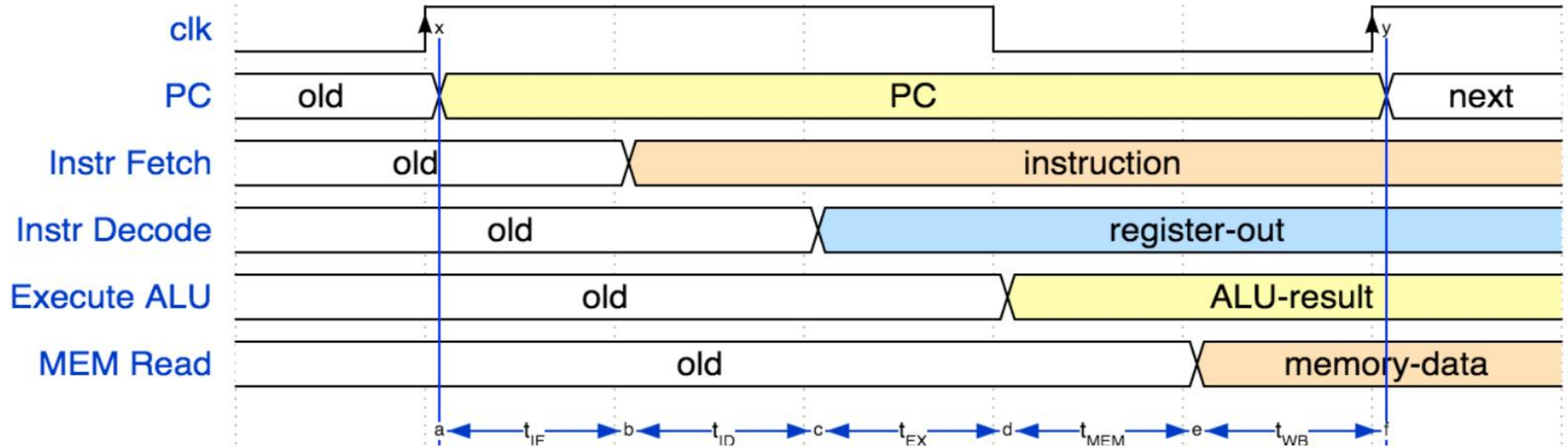
从信息流视角看CPU执行指令的基本过程



数据通路的五个阶段

- 取指: Instruction Fetch (IF)
- 译码: Instruction Decode (ID)
- 执行: EXecute (EX) —— ALU (Arithmetic-Logic Unit)
- 访存: MEMory access (MEM)
- 写回: Write Back to register (WB)

指令时延



IF	ID	EX	MEM	WB	Total
I-MEM	RegRead	ALU	D-MEM	RegWr	
200ps	100ps	200ps	200ps	100ps	800ps

指令时延

Instr	IF=200ps	ID=100ps	ALU=200ps	MEM=200ps	WB=100ps	Total
add	√	√	√		√	600ps
beq	√	√	√			500ps
jal	√	√	√		√	600ps
load	√	√	√	√	√	800ps
store	√	√	√	√		700ps

- 最大时钟频率 $f_{\max, \text{ALU}} = 1/(800\text{ps}) = 1.25\text{GHz}$
- 有些段多数时间处于空闲状态
- 单个段最大时钟频率 $f_{\max, \text{ALU}} = 1/(200\text{ps}) = 5\text{GHz}$

分阶段的数据通路——概述

- 问题：单块的逻辑结构完成对所有指令从取指到执行的所有操作，设计笨重，效率低下
- 分阶段的数据通路设计：（分而治之）将原来执行一条指令的过程，拆分为几个阶段，然后将这几个阶段对应的电路结构前后串联起来构成完整的数据通路。
 - 电路规模更小，更便于设计实现
 - 便于修改优化一个阶段而不影响其他阶段

单周期CPU处理器设计小结

- 实现了一个RISC V处理器数据通路
 - 能够在**一个时钟周期内**执行给定RISC-V指令子集
 - 并非所有指令都会用到所有硬件单元
 - **关键路径**
- 控制器指定如何执行指令
 - 基于ROM或组合逻辑实现
- 5个执行阶段（IF、ID、EX、MEM、WB）
 - 有的阶段只有部分指令才会用到

学习本课程的收获

- 高级语言程序如何被翻译成机器语言？
- CPU原理:硬件如何执行最终的程序？ 软件和硬件之间的接口是什么？
- 什么因素决定了程序的性能？ 程序员如何改进程序性能？
- 硬件设计人员可以使用哪些技术来提高性能？
 - 现代计算机设计的基本概念和原理——流水线、层次结构等
- 哪些伟大思想奠定了现代计算技术的基础？